# 軟體工程 (Software Engineering)

## 第五課：軟體設計、架構與物件導向原則 (Software Design & Architecture)

### 從通用設計基石、架構推導塑模到 SOLID 匠藝

**薛念林 教授**
資訊工程學系
逢甲大學

# 第五章：課程藍圖與核心主題

## 第一部分：設計流程、原則與巨觀設計

- **5.1 軟體設計導論、流程與設計規格：** 四階段軟體設計流程 (Arch → Interface → OO → Data) 與 SDD / ADR 規格。
- **5.2 通用軟體設計核心原則：** 抽象化、封裝、關注點分離、模組化，以及內聚力與耦合度的層次光譜。
- **5.3 階段一 — 架構設計 (Architectural Design)：** 六大架構模式、外送平台複合合成、ATAM 與 **UML 套件圖 / 部屬圖**。
- **5.4 階段二 — 介面與契約規格 (Interface & Contract)：** **契約式設計 (DbC)** 與 **UML 組件圖** (棒棒糖與插座)。

## 第二部分：微觀物件導向、資料設計、AI 與統整

- **5.5 階段三 — 組件與物件導向設計 (OO Design)：** GoF 23 種設計模式與五大 SOLID 原則之 **UML 類別圖** 對照。
- **5.6 階段四 — 資料與持久化設計 (Data Design)：** 三層次資料塑模、**UML ERD**、3NF 正規化、Redis 快取與多模態儲存。
- **5.7 AI 賦能軟體設計與盲點防護：** 橫跨四階段的 AI 設計副駕駛與人機協同防護。
- **5.8 核心概念統整與參考文獻：** 全章多維度總結評量矩陣、經典文獻與研討交流。

# 5.1 軟體設計導論、流程與設計規格

> *"構建軟體設計有兩種方法：一種是讓它簡單到明顯沒有缺陷，另一種是讓它複雜到沒有明顯的缺陷。"*
> *— C. A. R. Hoare (圖靈獎得主)*

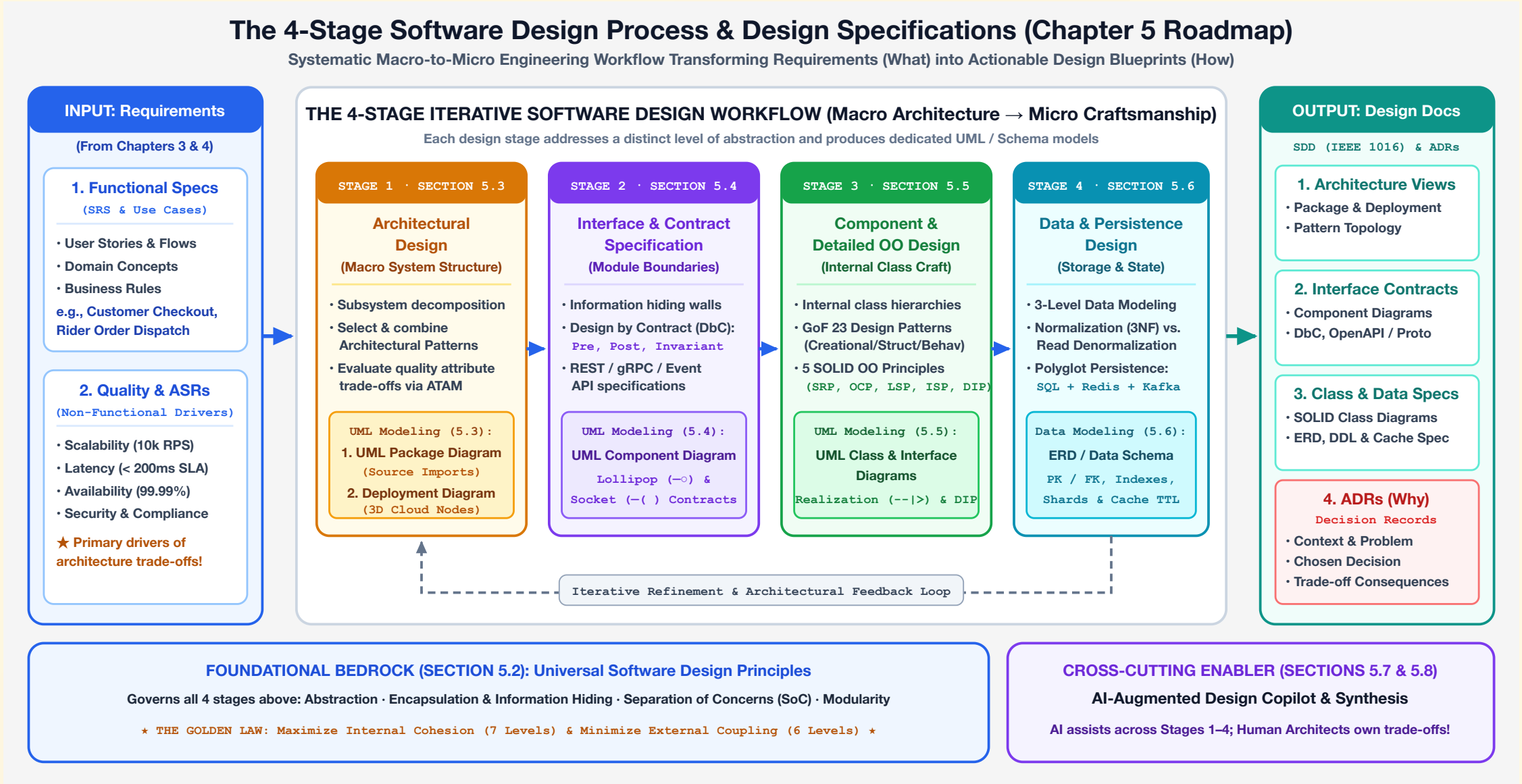
# 什麼是軟體設計？ (What is Software Design?)

- **從需求 (What) 到實作 (How) 的轉化過程：**
  - **需求工程 (Requirements / SRS)**：定義利益關係人的痛點、使用案例與品質期待（*建造什麼*）。
  - **軟體設計 (Software Design / SDD)**：構思結構分解、介面合約、類別階層與資料綱要（*如何建造*）。
- **軟體設計的核心任務：**
  - **建立工程規格與藍圖 (Blueprinting)**：產出架構模型、介面契約與資料庫綱要，讓工程團隊得以平行開發。
  - **進行權衡決策 (Trade-off Decision Making)**：在互斥的非功能需求間取得平衡（如低延遲 vs. 強一致性 vs. 雲端成本）。
- **「設計」與「寫程式 (Coding)」的本質區別：**
  - 寫程式是用特定程式語言語法實現演算法邏輯。
  - 設計則是決定**模組如何切割**、**介面契約如何規範**、**資料狀態如何持久化**以因應未來的變更。

# 為什麼軟體設計至關重要？ (Why Design Matters?)

- **1. 杜絕「憑感覺寫代碼 (Vibe Coding)」與結構腐敗：**
  - 缺乏設計的程式碼會迅速誘發 Robert C. Martin 指出的「不良設計四大病徵」：
    - **僵直性 (Rigidity)**：改動一處引發數十個無關檔案骨牌式連鎖修改。
    - **脆弱性 (Fragility)**：修復模組 A 的錯誤，莫名其妙導致無關的模組 B 崩潰。
    - **不可移植性 (Immobility)**：組件糾纏不清，無法被抽取復用於其他系統。
    - **黏滯性 (Viscosity)**：照正規設計修改比硬湊 Hack 更費力，誘使工程師走捷徑。
- **2. 品質屬性是「設計」出來的，不是「測試」出來的：**
  - 可擴展性、容錯力、低延遲與安全性必須從第一天納入架構骨幹，無法靠日後補寫單元測試補救。
- **3. Barry Boehm 缺陷修復成本曲線 (Cost of Change)：**
  - 在**需求與設計階段**修復瑕疵只需 **1 美元**；若拖到**正式上線維護階段**，修復代價飆升高達 **50 到 100 倍**！

# 四階段軟體設計流程與軟體設計文件 (SDD & ADR) 全景圖



# 四階段軟體設計流程與交付設計產物 (SDD & ADR)

> *第五章展開主軸：先掌握底層的通用設計原則 (5.2)，再依序執行四階段設計流程 (5.3 → 5.6)，並為每一階段搭配專屬的 UML 圖表。*

## 1. 四階段設計流程與專屬 UML 圖表

1. **階段一 — 架構設計 (5.3 節)**：
   - 組合架構模式 (Layered、微服務)、執行 ATAM 權衡，並繪製 **UML 套件圖與部屬圖**。
2. **階段二 — 介面與契約規格 (5.4 節)**：
   - 定義 **契約式設計 (DbC)** (`Pre`, `Post`, `Invariant`)，並繪製 **UML 組件圖** (`-o` / `-(`)。
3. **階段三 — 組件與 OO 設計 (5.5 節)**：
   - 運用 GoF 23 種設計模式與 **SOLID 五大原則**，並繪製 **UML 類別與介面圖**。
4. **階段四 — 資料與持久化設計 (5.6 節)**：
   - 設計三層次 **UML ERD 資料綱要**、3NF 正規化、Redis 空間快取與多模態儲存。

## 2. 軟體設計文件 (SDD 與 ADR 產物)

- **軟體設計文件 (Software Design Document, SDD — IEEE 1016)**：
  - **賦能平行開發**：前端、後端、DBA 與 QA 團隊依據鎖定的規格同步開發與撰寫測試，不必苦等實作完成。
  - **涵蓋內容**：套件/部屬架構圖、組件 API 合約 (OpenAPI/Protobuf)、類別模型與 DDL 綱要。
- **架構決策記錄 (ADR — Michael Nygard)**：
  - 納入 Git 版本控制的輕量文檔（記錄 `背景脈絡 Context`、`決策 Decision` 與 `正反面代價 Consequences`），保存架構取捨背後的 **Why**，杜絕「架構考古學」！

# 5.2 通用軟體設計核心原則

> *"優良的設計極易修改；拙劣的設計在每一步都抗拒改變。"*
> *— Dave Thomas (The Pragmatic Programmer)*

# 通用原則：貫穿全尺度的「分形 (Fractal)」特性

- **軟體設計具有分形特性 (Fractal Nature)：**
  - 同一套核心法則，在不同層級的放大鏡下展現出完全一致的規律：
    - **系統巨觀層級 (System Scale)**：微服務架構將業務領域拆解為獨立解耦、高凝聚的自治服務。
    - **模組套件層級 (Module Scale)**：Package 與函式庫將特定職責封裝於公開 API 邊界之後。
    - **類別微觀層級 (Class Scale)**：單一類別嚴格封裝狀態，只保留單一的改變理由 (SRP)。
- **原則的本質是為了克服人類心智的極限：**
  - 電腦記憶體有幾十 GB，但人類的「工作記憶 (Working Memory)」只有 7 ± 2 個區塊 (Miller's Law)。
  - 設計原則是工程心智工具，旨在確保大型軟體能被人類大腦理解、維護與擴充。

# 經典設計準則：模組化、抽象化與關注點分離

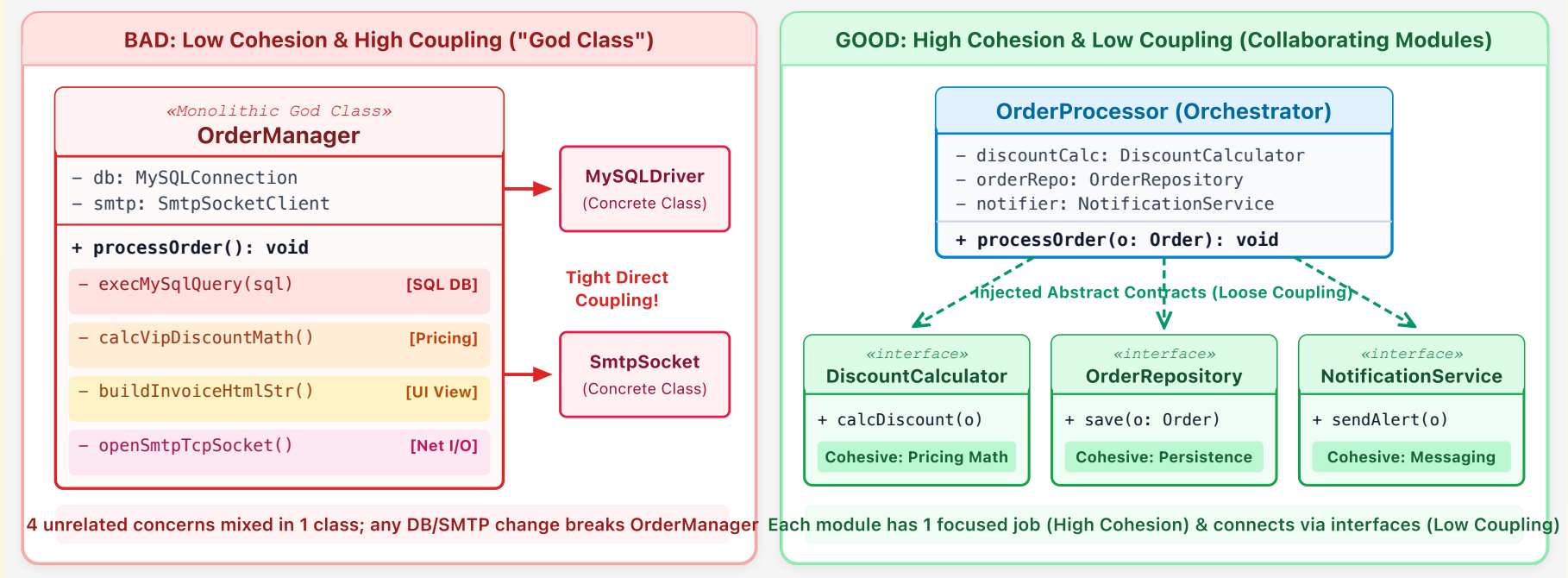
- **1. 抽象化 (Abstraction) — *[認知簡化視角：定義「做什麼 What」]*：**
  - 萃取核心概念並建立高階公開介面合約 (`interface`)，省略無關實作細節，降低開發者的心智認知負荷。
- **2. 封裝與資訊隱藏 (Encapsulation) — *[防護邊界視角：守護「怎麼做 How」]*：**
  - *David Parnas 原則：* 將狀態與行為綑綁於模組膠囊內並鎖住 `private` 內部變數，防止外部代碼破壞狀態不變性。
- **3. 關注點分離 (Separation of Concerns, SoC) — *[邏輯維度：正交切分不同職責面向]*：**
  - 將系統依不同問題維度正交拆解（如：UI 呈現 vs. 業務規則 vs. SQL 查詢 vs. 日誌稽核），避免不同面向糾纏。
- **4. 模組化 (Modularity) — *[實體封裝：打造可獨立測試與插拔的結構單元]*：**
  - 在實體結構上將系統組織為邊界清晰、高內聚的獨立建構單元（如 Package、JAR、微服務），支援平行開發與熱替換。

# 至高黃金法則：高凝聚度與低耦合度

- **凝聚度 (Cohesion)：**
  - 衡量單一類別或模組內部所有責任與成員彼此關聯的緊密程度。
  - **工程目標： 高凝聚度 (High Cohesion)**（一個模組只專注做好一件事）。
- **耦合度 (Coupling)：**
  - 衡量不同模組之間直接相依與牽扯的強烈程度。
  - **工程目標： 低耦合度 (Low Coupling)**（模組間透過抽象合約互動，對彼此實作細節完全無知）。
- **軟體工程至高黃金法則：**

> *「極大化凝聚度，極小化耦合度！」*
> *高凝聚確保模組專注好懂；低耦合確保改動不會引發骨牌式連鎖崩潰。*

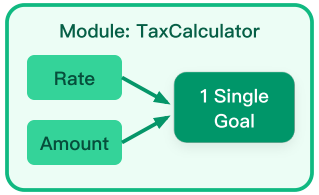
# 凝聚度與耦合度：不良設計 vs. 優良設計對比 (UML 視角)




- **劣質設計 (`OrderManager` 上帝類別)**：單一類別塞入 SQL、折扣運算、HTML 與 SMTP 連線，導致**偶然性低內聚與緊密實體耦合**。
- **優良設計 (`OrderProcessor` 協同架構)**：各類別專注單一職責（**功能性高內聚**），並透過注入抽象介面協作（**鬆散介面解耦**）。

# 內聚力視覺化譜系：7 大內聚層次圖解 (從強到弱)

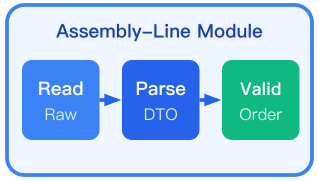
## Strong & Focused Cohesion (Levels 7 → 4)




### 7. Functional Cohesion
(1 Single Well-Defined Task)

IDEAL (BEST)

DeliveryFeeCalc
computeFee(distKm)
100% Focused on 1 Math Rule

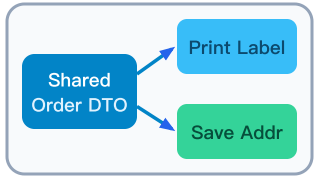



### 6. Sequential Cohesion
(Output of A Feeds Input of B)

STRONG

1. readOrderJson()
2. validateAndTotal()

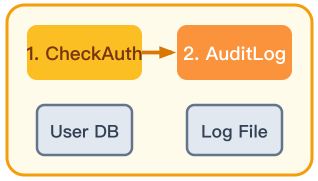
Direct Data Relay Pipeline




### 5. Communicational
(Operate on Same Data Record)

ACCEPTABLE

CustomerRecord
updateAddr()
printReceipt()

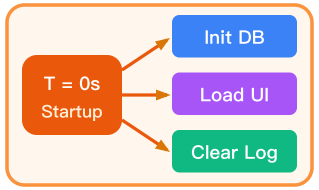



### 4. Procedural Cohesion
(Grouped by Order, Diff Data)

MODERATE

Step 1: checkPermission()
↓ (Control Order Only)
Step 2: writeDiskAudit()

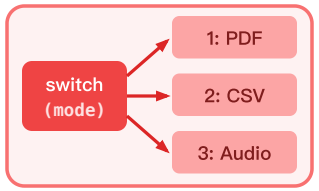
## Weak & Entangled Cohesion (Levels 3 → 1)




### 3. Temporal Cohesion
(Same Time Slot Only)

WEAK (REFACTOR)

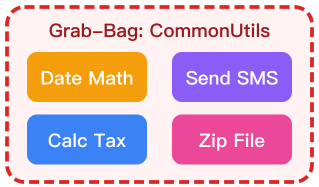
initStartup()
connectDb() + renderSplash()
+ pingSmtp() + loadFonts()
Unrelated Startup Batch




### 2. Logical Cohesion
(Flag-Controlled Switch Box)

POOR (ANTI–OCP)

export(int flag)
if (flag==1) drawPdf();
else if (flag==2) dumpCsv();
Tangled Multi–Branch Logic




### 1. Coincidental Cohesion
(Zero Meaningful Relation)

WORST (ELIMINATE)

MiscHelper.java
formatCurrency() // UI
openTcpSocket() // Net
computeVatTax() // Biz

**Cohesion Refactoring Rule (Single Responsibility Principle)**

Split Coincidental / Logical / Temporal God Classes (Levels 1–3)

→ Into Focused Functional & Sequential Collaborators (Levels 6–7)

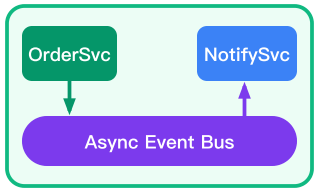
# 內聚力的層次與類型：從弱到強 (Weak to Strong)

- **內聚力譜系 (Yourdon & Constantine)**：衡量單一模組內部各元素彼此關聯與聚焦的緊密程度：

| 層次 | 內聚力類型 | 核心特徵與運作機制 | 實務範例 | 品質評估 |
|---|---|---|---|---|
| 7 | **功能性 (Functional)** | 所有元素只為完成單一、明確的業務或運算目標 | `calculateTax()`, `sin(x)` | **最高境界** |
| 6 | **順序性 (Sequential)** | 前一步驟的輸出直接作為下一步驟的輸入（流水線） | `read() → parse() → validate()` | **強烈推薦** |
| 5 | **溝通性 (Communicational)** | 各項操作共用相同的輸入資料或產出相同輸出 | `updateAddr()` 兼 `printLabel()` | **可接受** |
| 4 | **過程性 (Procedural)** | 僅因特定執行先後順序而綁在一起，處理不同資料 | 先 `checkAuth()` 再 `auditLog()` | **普通 (中度)** |
| 3 | **時間性 (Temporal)** | 元素僅因「在同一時間點執行」而被包在一起 | `initStartup()`, `closeAll()` | **偏弱 (應重構)** |
| 2 | **邏輯性 (Logical)** | 包攬同類任務，由外部傳入的 Flag 決定執行分支 | `export(int mode)` (CSV/PDF) | **不良 (違 OCP)** |
| 1 | **偶然性 (Coincidental)** | 元素間毫無邏輯關聯，純屬隨機拼湊的大雜燴 | `CommonUtils`, `MiscHelper` | **最差 (杜絕)** |

# 耦合度視覺化譜系：6 大耦合層次圖解 (從鬆到緊)

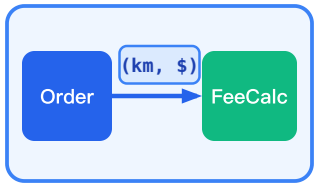
## Loose & Modular Coupling (Levels 6 → 4)




### 6. Message / Event
(Async Events, Zero Direct Ref)

IDEAL (LOOSEST)

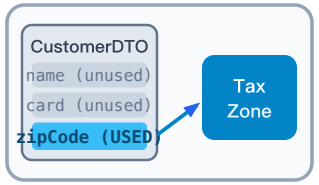
```
publish(OrderPlaced)
```
Sender & Receiver Have
0 Compile–Time Dependency




### 5. Data Coupling
(Passes Needed Primitives Only)

BEST (DIRECT)

```
calcFee(double km)
```
Passes Only Exact Values
Zero Knowledge of Order
or Customer Class Structure




### 4. Stamp Coupling
(Whole Object for 1 Field)

MODERATE

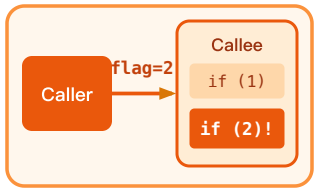
```
findZone(Customer c)
return map.get(c.zipCode);
```
Risk: Tied to Customer Class
Fix: Pass String zipCode

**Loose Coupling Firewall (Levels 5–6 + Interface Contracts)**

Module A → [Narrow Interface / Primitives / Event] → Module B

Result: Changing Module B Internals Causes Zero Ripple Effect on Module A

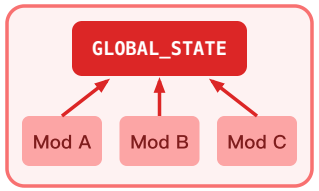
## Tight & Pathological Coupling (Levels 3 → 1)




### 3. Control Coupling
(Flag Dictates Internal Branch)

HIGH RISK

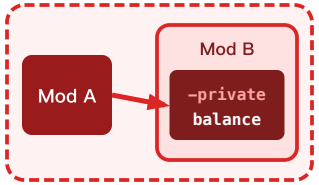
```
pay(amt, int mode)
```
Caller Must Know Internal
if/else Flag Constants
Fix: Polymorphic Strategy




### 2. Common (Global)
(Shared Global Mutable State)

SEVERE RISK

```
GlobalApp.currentCart
```
Hidden Side Effects &
Thread Race Conditions
Any Reader Can Corrupt All




### 1. Content Coupling
(Mutates Private Internals)

WORST (PATHOLOGICAL)

```
Encapsulation Breach
order.payment.rawSql
= "UPDATE paid=1";
```
Any Change in B Breaks A!

**Tight Coupling Ripple Effect (Levels 1–3 Domino Failure)**

Change 1 Private Field in Mod B → Breaks Mod A → Breaks Mod C

→ Refactor to Data / Event Coupling (Levels 5–6) via Narrow Interfaces

# 耦合度的層次與類型：從緊到鬆 (Tight to Loose)

- **耦合度譜系**：衡量不同模組之間呼叫、資料傳遞與知識共享的牽扯深度：

| 層次 | 耦合度類型 | 依賴機制與破壞風險 | 實務範例 | 品質評估 |
|---|---|---|---|---|
| 1 | **內容耦合 (Content)** | 直接存取或竄改另一模組的私有內部欄位或分支 | 繞過封裝直接改他者 `private` 欄位 | **最差 (極危險)** |
| 2 | **共用耦合 (Common)** | 多個模組共同讀寫全域可變變數或共享狀態 | 多執行緒共用未保護的全域 Flag | **高風險** |
| 3 | **控制耦合 (Control)** | 透過傳遞控制旗標 (Flag) 支配另一模組內部流程 | 傳入 `mode = 3` 操控被呼叫者分支 | **中高風險** |
| 4 | **標記耦合 (Stamp)** | 傳遞整包複合資料物件，但對方僅需其中一欄 | 只需 `zipCode` 卻傳入整包 `Customer` | **中度 (可改善)** |
| 5 | **資料耦合 (Data)** | 僅透過簡單、獨立且必要的純量參數進行通訊 | `calculateArea(width, height)` | **呼叫最佳** |
| 6 | **訊息/事件 (Message)** | 透過非同步事件通訊，彼此完全不知曉對方存在 | EventBus 發布 `OrderPlacedEvent` | **最理想 (解耦)** |

# 觀念檢核測驗 1 (CCQ 1)

在軟體系統設計中，將**高凝聚度 (High Cohesion)** 與 **低耦合度 (Low Coupling)** 相互結合，其最主要的實質工程效益為何？

- **A.** 將所有輔助函式全部內聯至單一執行檔中，極大化處理器底層指令執行速度。
- **B.** 將內部實作變動隔離於模組邊界內，使單一類別修改不致引發跨模組連鎖損壞。
- **C.** 強制規定將所有領域資料結構集中存放於單一全域可變記憶體空間中統一管理。
- **D.** 徹底免除在持續整合管線中撰寫與執行自動化整合測試及回歸測試套件的負擔。



# 5.3 階段一 — 架構設計：架構模式、ATAM 權衡與套件/部屬圖

> *「軟體架構沒有完美的解法，只有持續的權衡取捨。」*
> *— Thomas Sowell*

# 什麼是軟體架構？ (What is Software Architecture?)

- **正式定義 (IEEE 1471 / ISO 42010)：**

> *軟體架構是系統的根本性組織，體現在其組件、組件彼此與環境之間的相互關係，以及指導其設計與演進的原則中。*

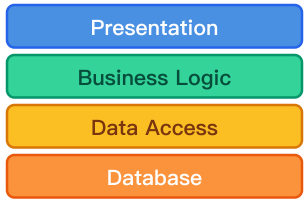
- **架構的「橋樑角色 (The Bridge Role)」：**
  - **需求 (What)**：利益關係人的期望、使用者故事與限制條件。
  - **實作 (How)**：具體程式碼、資料表綱要與測試案例。
  - **架構 (橋樑)**：將混亂的人類需求轉化為結構化的子系統、服務邊界與部屬拓撲。
- **微觀架構 vs. 巨觀架構 (In the Small vs. In the Large)：**
  - **微觀架構 (In the Small)**：單一應用程式內部的模組組織與類別劃分。
  - **巨觀架構 (In the Large)**：企業級分散式系統、雲端叢集、事件匯流排與異質資料庫拓撲。

# 什麼是軟體架構模式？ (Architectural Patterns)

- **架構模式的本質：**
  - 架構模式是在給定軟體工程情境下，針對**反覆出現的結構性設計問題，所提出經過實戰檢驗的通用可復用解法**。
  - 設計模式 (Design Patterns) 作用於微觀類別層次；架構模式則決定**整個系統的巨觀宏觀結構拓撲**。
- **架構模式的三大核心要素：**
  - **1. 情境 (Context)**：特定反覆出現的軟體系統開發環境與工程挑戰。
  - **2. 問題與作用力 (Problem & Forces)**：相互抗衡的需求拉扯（如：資料解耦 vs. 即時性延遲）。
  - **3. 解法與代價 (Solution & Consequences)**：組件劃分、通訊約束與必須接受的缺點妥協。
- **為什麼架構模式至關重要：**
  - 為大型工程團隊建立共通詞彙，避免重複犯下歷史上的經典架構錯誤。

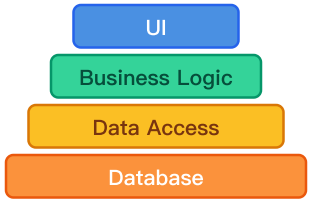
# 架構模式視覺導覽：5.3 六大經典架構樣式總覽

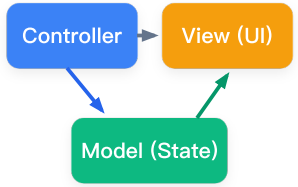
## Centralized & Structural Patterns




### 2. Layered (N-Tier) Architecture

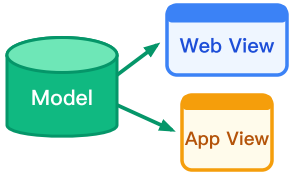
(Strict Downward Calls)

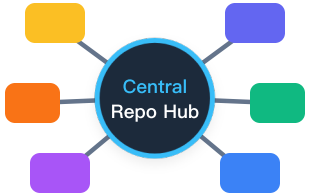



### 1. Model-View-Controller (MVC) Architecture

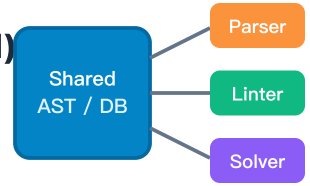
(Decoupled UI & State)

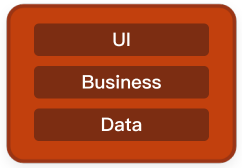


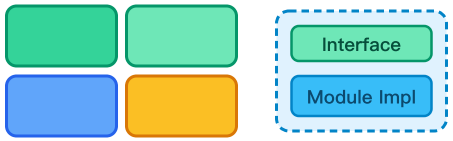

### 3. Repository (Blackboard) Architecture

(Central Shared Data Store)




Monolithic Architecture



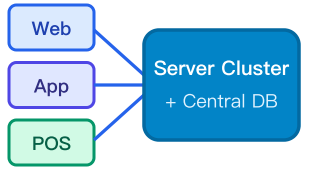

Modular Component Design

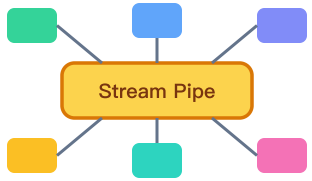
## Distributed, Routing & Flow Patterns



### 4. Client-Server Architecture

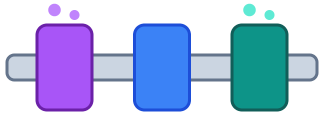
(Edge Clients ↔ Central Tier)

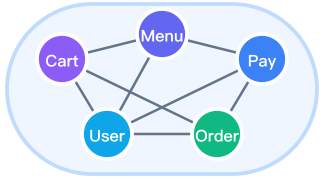



### 5. Pipe-and-Filter Stream Architecture

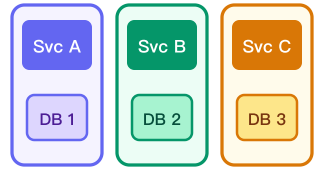
(Sequential Stateless Filters)

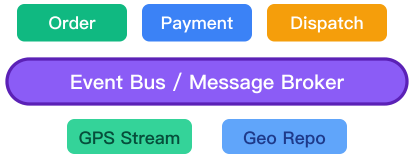



### 6. Microservices (MSA) Architecture

(Database-per-Service Pods)




Event–Driven Bus Topology

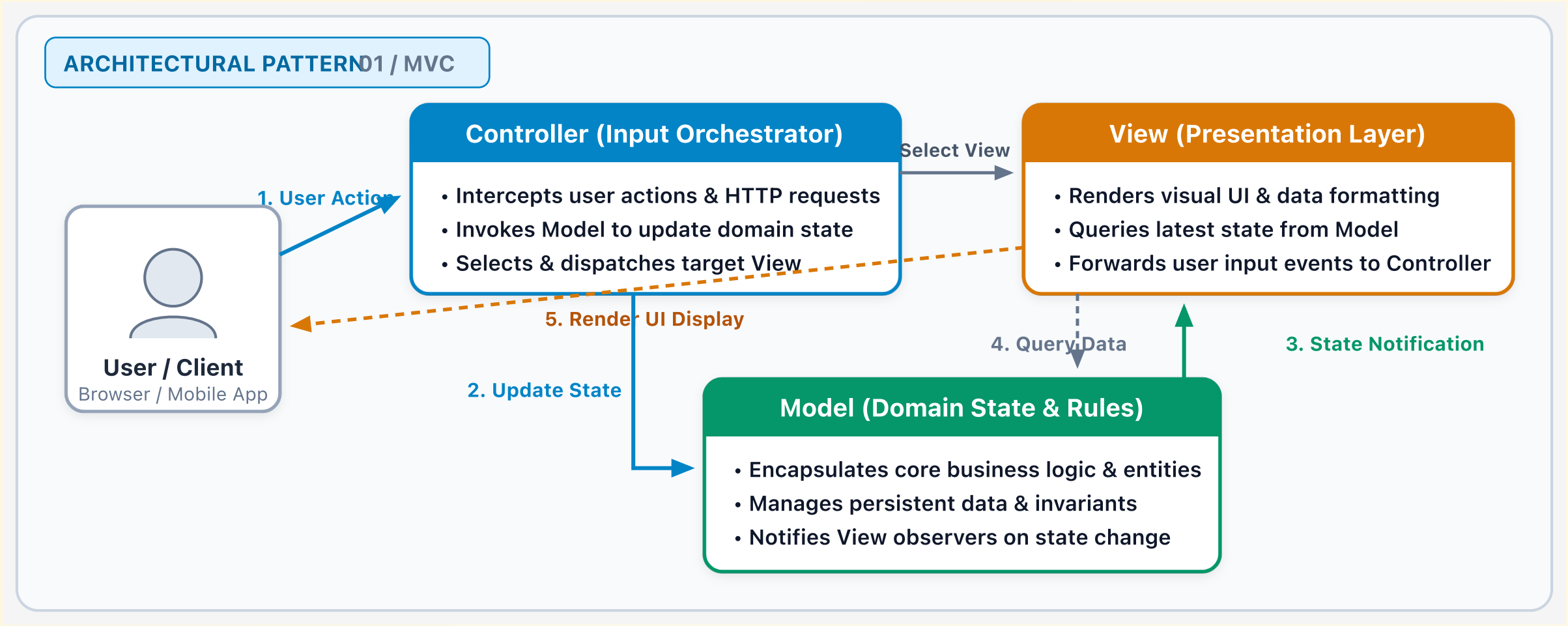
Real–World System Composition
Client–Server + MVC (Frontend)
+ Microservices + Layered (Services)
+ Pipe–Filter + Repository (Dispatch)

Multi–Pattern Hybrid Architecture

# 六大經典架構模式全景對照總表

| 模式名稱 | 拓撲外觀特徵 | 核心運作機制 | 最適用場景 | 主要風險與承擔代價 |
|---|---|---|---|---|
| **1. MVC** | 三角循環迴圈（`C → M → V`） | 徹底分離使用者介面與底層領域資料模型 | 具備豐富互動界面的 Web / 行動應用 | 對單純資料展示 (CRUD) 過於繁瑣複雜 |
| **2. 分層架構** | 垂直堆疊階層（`N → N-1`） | 嚴格由上至下單向呼叫，跨層抽象隔離 | 企業級資訊系統、傳統關聯式資料庫應用 | 呼叫堆疊過深引發效能耗損（水槽反模式） |
| **3. 儲存庫架構** | 星狀輻射樞紐架構 | 主動協同組件圍繞被動共用資料庫 | 編譯器、IDE 開發環境、多工資料分析系統 | 中央資料庫成為效能瓶頸與單一故障點 (SPOF) |
| **4. 主從式架構** | 跨網路分散式（`C ↔ S`） | 前端客戶端向集中式後端伺服器請求資源 | Web 網站、檔案伺服器、集中管控服務 | 網路延遲風險；尖峰流量伺服器過載 |
| **5. 管線過濾器** | 線性單向資料流串接 | 獨立過濾器藉由單向管線串接串流數據 | 影音轉檔、大數據 ETL、編譯詞法分析 | 不適用於互動式 GUI；相鄰介面資料轉換開銷 |
| **6. 微服務架構** | 去中心化服務網狀拓撲 | 獨立部署自治微服務，各擁專屬私有 DB | 超大型、團隊規模龐大、需極速迭代的雲平台 | 分散式系統複雜度：網路延遲、分散式交易 |

# 1. 模型-視圖-控制器模式 (MVC) — 架構拓撲圖



# 1. 模型-視圖-控制器模式 (MVC)：職責與權衡

- **三大核心組件職責切分：**
  - **Model (模型)**：封裝核心領域商業資料、驗證規則與真實狀態，對前端視覺呈現完全無知。
  - **View (視圖)**：負責將資料格式化並渲染成視覺介面（HTML、App 畫面、圖表），監聽模型狀態變化。
  - **Controller (控制器)**：攔截使用者輸入與 HTTP 請求，轉譯並調度商業邏輯以更新 Model，並選派對應 View。
- **適用場景與實務案例：**
  - 具備豐富使用者互動、或同一份領域資料需同時支援多種視覺呈現方式（如 Web 儀表板、行動介面、匯出報表）的系統。
- **工程優勢與取捨代價：**
  - **優勢**：徹底落實關注點分離；前端 UI 設計師與後端工程師可平行開發、獨立演進而不互相干擾。
  - **代價**：對極簡單的單純資料展示 (CRUD) 略顯繁瑣；若 View 繞過 Controller 直接修改狀態會破壞解耦邊界。

# 2. 分層架構模式 (Layered Architecture) — 架構拓撲圖

ARCHITECTURAL PATTERN 02 / LAYERED

**Presentation**
UI / API Layer
Web Views, Mobile UI, REST / GraphQL Controllers

**Application**
Service Layer
Use-Case Orchestration, Auth, Transaction Boundaries

**Business / Domain**
Core Logic Layer
Domain Entities, Business Rules, Core Algorithms

**Persistence**
Infrastructure
Repositories, SQL/NoSQL Databases, External Adapters

### Architectural Rules

**Strict Downward Calls:**
Layer N only calls Layer N-1; no upward or skipped-layer calls.

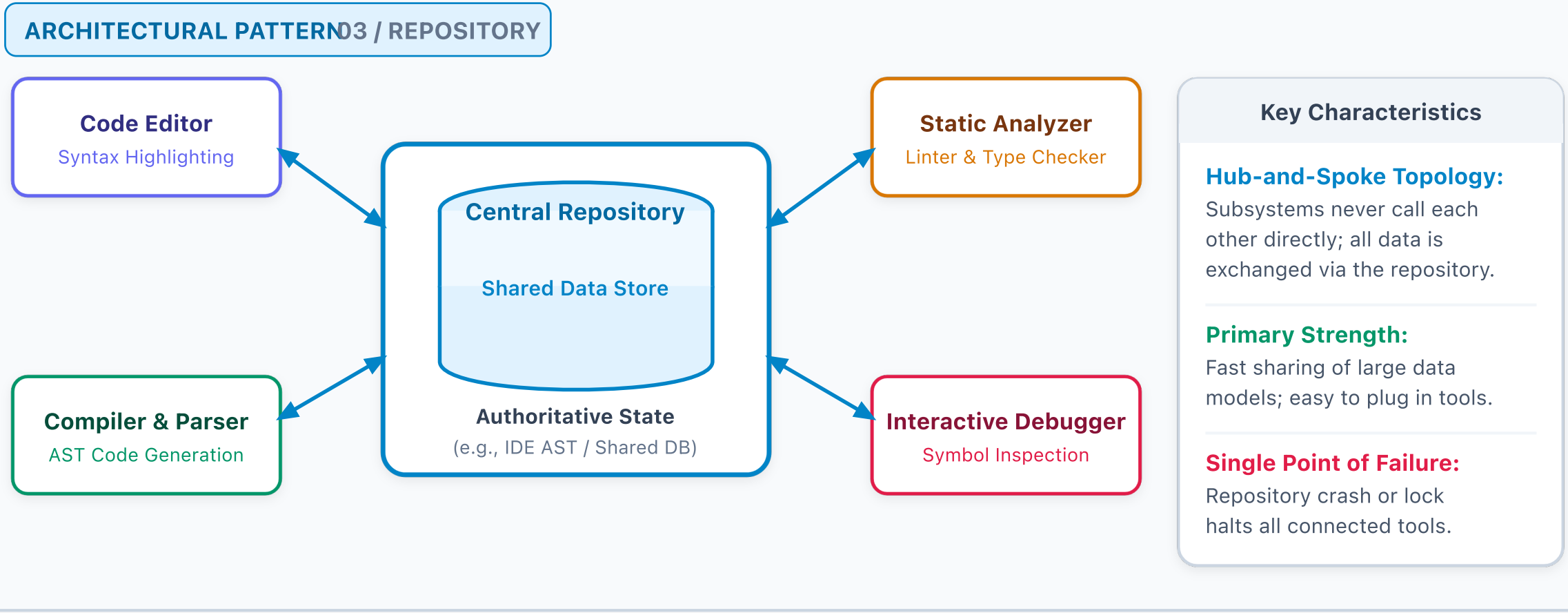
**Change Isolation:**
Swapping MySQL for DynamoDB only touches Persistence; core Domain Logic stays untouched.

**Sinkhole Anti-Pattern:**
Pass-through requests across all layers add latency overhead.

# 2. 分層架構模式 (Layered Architecture)：職責與權衡

- **階層劃分與相依規則：**
  - 水平劃分為四層：**展示層 (Presentation)** → **應用服務層 (Application)** → **領域業務層 (Business)** → **持久化層 (Persistence)**。
  - **嚴格分層原則 (Strict Layering)**：每一層僅相依於緊鄰的下一層 (`Layer N → Layer N-1`)，嚴禁由下往上逆向呼叫或越級跨層存取。
- **適用場景與實務案例：**
  - 標準企業資訊系統、電商後台、金融帳務核心系統等需要明確團隊分工與多層級資安防護的應用。
- **工程優勢與取捨代價：**
  - **優勢**：抽象職責清晰、變更隔離極佳——若底層資料庫由 MySQL 遷移至 DynamoDB，核心領域業務層完全零衝擊。
  - **代價**：易陷入**架構水槽反模式 (Architecture Sinkhole Anti-pattern)**（大量請求僅層層穿透轉發而無實質商業運算，徒增呼叫與物件轉換延遲）。

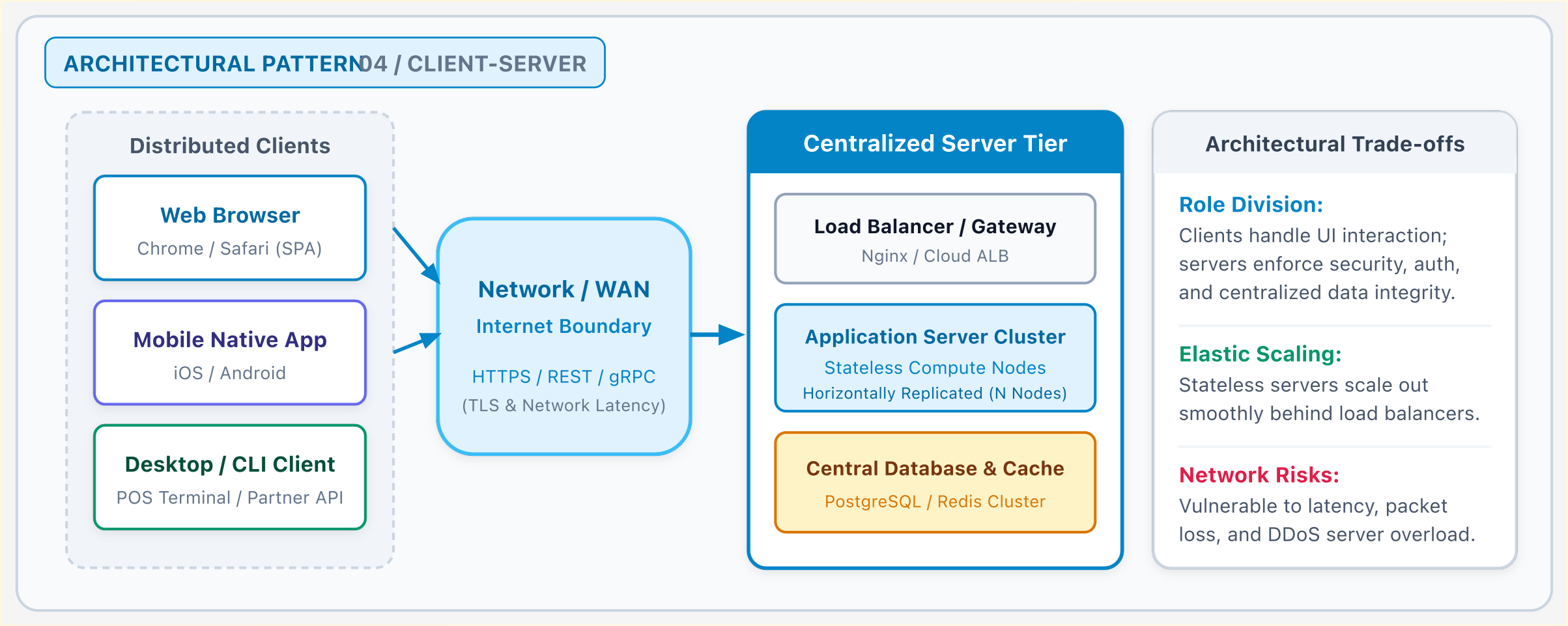
# 3. 儲存庫架構模式 (Repository Architecture) — 架構拓撲圖



# 3. 儲存庫架構模式 (Repository Architecture)：職責與權衡

- **核心組件與拓撲特徵：**
  - **中央儲存庫 (Central Repository)**：維護系統所有共享持久化狀態與資料一致性規則的被動儲存中心。
  - **獨立存取組件 (Active Subsystems)**：各自獨立運作的工具或分析子系統，彼此互不知曉，僅透過中央儲存庫讀寫共享資料。
- **適用場景與實務案例：**
  - 需要多個專業工具圍繞同一份龐大複雜資料結構協同處理的系統：**整合開發環境 (IDE)**、**編譯器 (共享 AST 語法樹)**、**CAD 工程繪圖系統**、**AI 黑板系統 (Blackboard)**。
- **工程優勢與取捨代價：**
  - **優勢**：徹底消除工具間的直接相依；新增外掛工具（如新語法檢查器）只需對接中央儲存庫，無須修改既有組件。
  - **代價**：中央儲存庫極易成為效能瓶頸與**單一故障點 (SPOF)**；一旦中央資料綱要 (Schema) 變更，所有相依子系統皆受影響。

# 4. 主從式架構模式 (Client-Server Architecture) — 架構拓撲圖

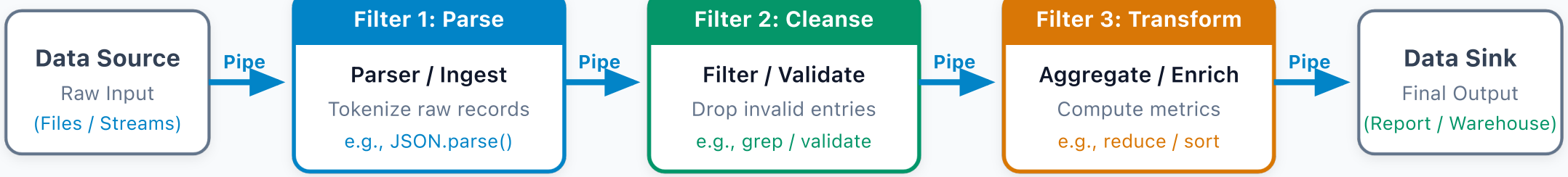


# 4. 主從式架構模式 (Client-Server Architecture)：職責與權衡

- **分散式節點職責分工：**
  - **客戶端 (Client Tier)**：分散於使用者終端的消費者，負責介面渲染、本地互動，並向伺服端發起網路服務請求。
  - **伺服端 (Server Tier)**：集中式服務提供者，負責權限驗證、核心商業邏輯運算、並行存取控制與資料持久化。
- **適用場景與實務案例：**
  - 現代 Web 網站、行動銀行 App、企業內部系統與線上多人服務——凡需讓多地終端安全存取共享資料的場景皆適用。
- **工程優勢與取捨代價：**
  - **優勢**：機密資料與資安防護集中於伺服端治理；後端升級對客戶端透明；無狀態伺服器可透過負載平衡器輕鬆水平擴展。
  - **代價**：高度受制於網路延遲、封包遺失與斷線風險；集中式伺服器端點面臨尖峰流量過載與 DDoS 攻擊威脅。

# 5. 管線與過濾器模式 (Pipe-and-Filter) — 架構拓撲圖

ARCHITECTURAL PATTERN 05 / PIPE & FILTER




Terminal (Unix Shell Pipe Analogy)

```
$ cat access.log | grep "500" | sort | uniq -c
```

Each filter is stateless and decoupled from upstream/downstream stages.

High Composability: Filters can be recombined or parallelized freely.

**Best Used For:** ETL pipelines, compilers, stream processing

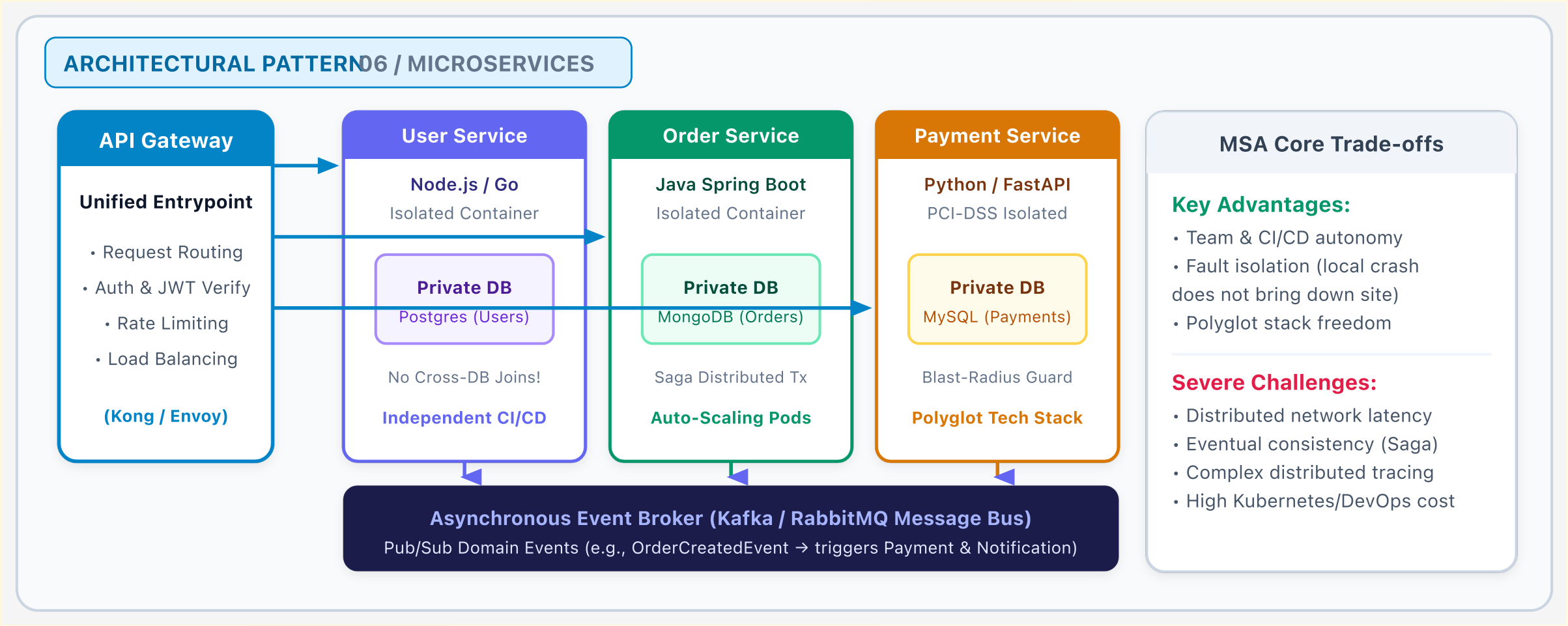
**Key Strength:** High reuse & easy multi-core concurrency

**Trade-off:** Unfit for interactive UIs; parsing overhead

# 5. 管線與過濾器模式 (Pipe-and-Filter)：職責與權衡

- **資料流串流機制：**
  - **過濾器 (Filters)**：獨立、無狀態 (Stateless) 的資料轉換單元，只負責讀取輸入串流、執行單一轉換並產出輸出串流，完全不知曉上下游組件是誰。
  - **管線 (Pipes)**：單向資料傳輸通道，負責緩衝並將資料流從上游過濾器傳遞至下游過濾器。
- **適用場景與實務案例：**
  - 多階段循序資料轉換系統：**Unix Shell 指令管線 (`cat | grep | sort`)、編譯器前端、影音串流編解碼管線、大數據批次 ETL 資料工程**。
- **工程優勢與取捨代價：**
  - **優勢**：極致的高內聚與可重組性（各過濾器可自由抽換、重排或復用）；各階段天然支援多核心並行運算加速。
  - **代價**：完全不適合需要即時雙向回饋的互動式 GUI 系統；相鄰過濾器之間反覆解析與序列化資料流會帶來效能開銷。

# 6. 微服務架構模式 (Microservices, MSA) — 架構拓撲圖



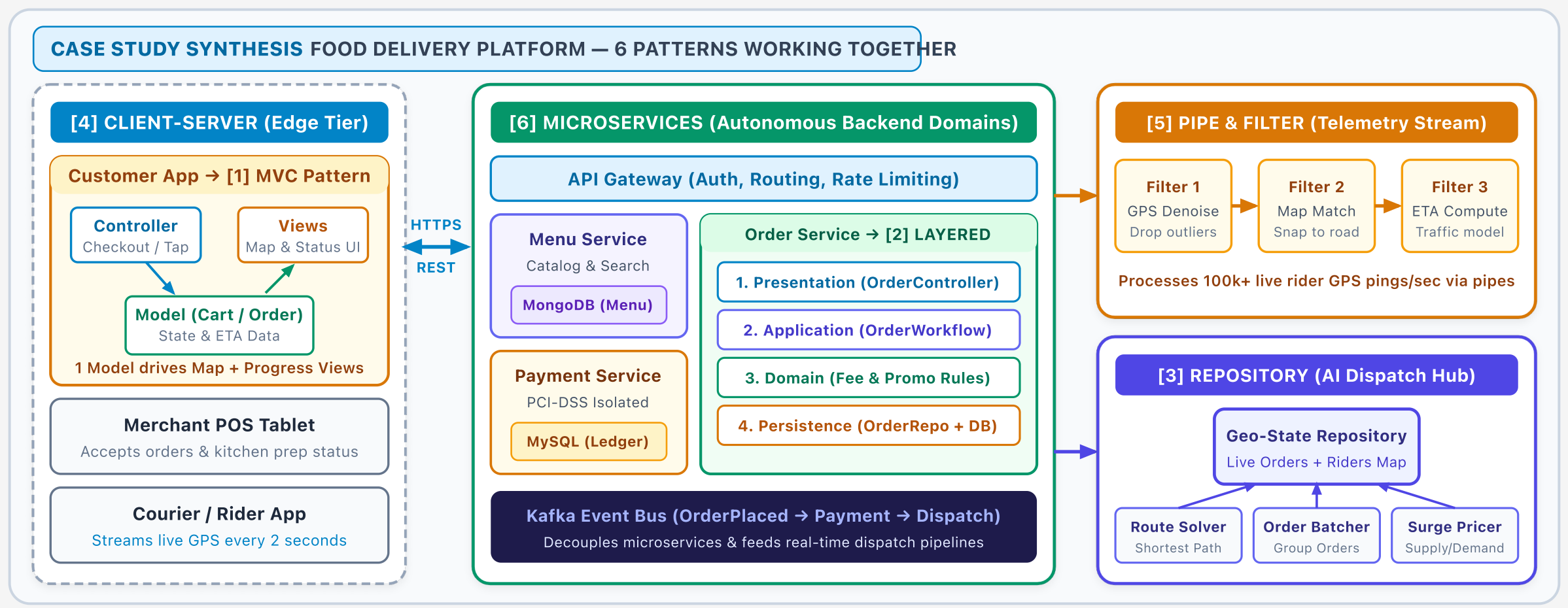
# 6. 微服務架構模式 (Microservices, MSA)：職責與權衡

- **去中心化自治服務核心原則：**
  - **獨立領域服務 (Autonomous Services)**：圍繞業務領域邊界 (Bounded Context) 拆分為小型服務，各自運行於獨立容器並具備專屬 CI/CD 部署流水線。
  - **一服務一資料庫 (Database-per-Service)**：各微服務嚴格封裝專屬私有資料庫；跨服務僅能透過 API (REST/gRPC) 或非同步事件匯流排 (Event Bus) 通訊。
- **適用場景與實務案例：**
  - 組織規模龐大（數十個敏捷開發團隊）、業務領域複雜且需高頻獨立上線與局部彈性擴展的大型雲端平台（如 Netflix、Uber、Amazon）。
- **工程優勢與取捨代價：**
  - **優勢**：**卓越的故障隔離（控制衝擊半徑 Blast Radius）**；支援單點瓶頸服務獨立擴展；團隊自主部署與異質技術棧自由選用。
  - **代價**：分散式系統複雜度飆升：網路延遲、跨庫分散式交易 (Saga 最終一致性)、分散式追蹤除錯困難，以及高昂的 Kubernetes/DevOps 維運成本。

# 單體架構 vs. 微服務架構：深度技術對比

| 比較維度 | 單體架構 (Monolithic Architecture) | 微服務架構 (Microservices Architecture) |
|---|---|---|
| **部署單元** | 單一巨大二進位執行檔 / WAR 包 / 容器 | 數十至數百個獨立運行的輕量微容器 |
| **資料儲存** | 單一共享全域關聯式資料庫 (Shared DB) | **各服務私有資料庫 (Database-per-Service)** |
| **通訊成本** | 極速奈秒級程序內記憶體呼叫 (In-memory) | 毫秒級網路呼叫 (RPC / REST / gRPC / 訊息佇列) |
| **故障隔離** | **極差**：模組 A 記憶體洩漏會導致全系統當機 | **卓越**：金流服務崩潰，使用者依然能正常瀏覽商品 |
| **團隊擴展** | 龐大代碼衝突、部署協調成本隨人數激增 | 獨立特徵團隊，每週自主上線部署數十次 |
| **維運門檻** | 低門檻（單一 CI/CD 流水線、集中式日誌） | 極高門檻（Kubernetes、分散式追蹤、服務網格） |
| **適用時機** | 新創公司、MVP 概念驗證、小型團隊 (<15人) | 組織規模龐大 (>50人)、業務領域極度複雜之企業 |

# 實戰案例：外送平台系統 (Food Delivery) — 複合架構全景圖



# 實戰案例解析：外送平台如何組合六大架構模式？

- **複合架構思維 (Multi-Pattern Composition)**：依**縮放層級 (Zoom Level)** 與**業務特性**組合模式：

| 架構模式 | 在 Food Delivery 系統中的對應部位與層級 | 解決的核心工程問題 |
|---|---|---|
| **1. MVC** | **顧客點餐 App 與商家接單看板 (前端互動層級)** | 同一份 `OrderModel` 狀態變更時，同步刷新「即時地圖 View」與「進度條 View」 |
| **2. 分層架構** | `Order Service` **服務內部 (** `Controller → Domain → Repo` **)** | 將運費計算、滿額折抵等核心商業規則與底層 PostgreSQL 資料庫徹底隔離 |
| **3. 儲存庫架構** | **AI 智慧派單引擎 (** `Geo-State Repository` **)** | 路徑規劃、順路併單、動態加價等演算法圍繞中央「全城即時座標狀態庫」協作 |
| **4. 主從式架構** | **三大終端 App (顧客/餐廳/外送員) ↔ 雲端叢集** | 將菜單定價、優惠驗證與金流扣款集中於伺服端管控，防止客戶端竄改金額 |
| **5. 管線過濾器** | **外送員即時 GPS 串流 (** `雜訊過濾 → 道路匹配 → ETA` **)** | 以無狀態管線平行處理全城每秒 10 萬筆以上 GPS 軌跡座標與送達時間預估 |
| **6. 微服務架構** | **雲端後台領域拆分 (** `Menu` **,** `Order` **,** `Pay` **,** `Dispatch` **)** | 暴雨尖峰可單獨將派單服務擴展 20 倍；評價服務當機絕不波及點餐結帳 |

# 軟體架構直接決定非功能品質屬性

- **品質屬性無法在軟體完工後靠補丁外掛：**
  - 你無法建造一個高度緊耦合的單體系統，然後指望靠寫幾個單元測試就讓它具備百萬並行能力。
- **架構決策如何實質主宰品質屬性：**
  - ⚡ **效能 (Performance)**：由跨網路呼叫跳轉數、記憶體快取層與資料庫分割策略直接決定。
  - 🛡️ **安全性 (Security)**：由零信任網路隔離、API 閘道防護與機密資料庫階層切分嚴格把關。
  - 🦺 **安全性/防護力 (Safety)**：由關鍵控制核心與非關鍵模組之物理隔離所保障（如汽車煞車 vs. 影音娛樂系統）。
  - 🔄 **可用性與容錯 (Availability)**：由冗餘備援節點、健康檢查、斷路器 (Circuit Breaker) 與自動容錯轉移所構築。
  - 🔧 **可維護性 (Maintainability)**：由系統是否嚴格落實高凝聚與低耦合邊界所主宰。

# 架構權衡分析 (Architecture Trade-off Analysis) 與 ATAM

- **不可避免的衝突矩陣 (No Silver Bullet)：**
  - **效能 vs. 安全性**：深層封包檢測、加密與防詐審核必然增加回應延遲。
  - **可用性 vs. 一致性 (CAP 定理)**：分散式強一致性鎖定會在尖峰網路抖動時阻塞結帳。
  - **模組化 vs. 執行效能**：細粒度分層與跨微服務 RPC 呼叫會增加序列化開銷。
- **ATAM (架構權衡分析法 — 卡內基美隆大學 SEI)：**
  - 在寫程式前，透過具體「品質屬性情境 (Scenarios)」系統化評估架構決策：
    - **敏感點 (Sensitivity Points)**：對單一品質屬性起決定性作用的關鍵決策（如：`Payment Service` 的斷路器直接決定結帳**高可用性**）。
    - **權衡點 (Trade-off Points)**：同時拉扯多個互斥品質屬性的決策點（顧此則失彼）。
    - **風險點與非風險點 (Risks & Non-Risks)**：尚未緩解的架構脆弱點 vs. 已驗證安全的設計。

# ATAM 實戰案例：外送平台 (Food Delivery) 架構權衡分析

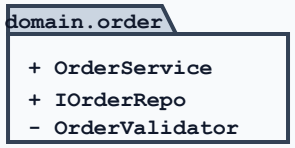
- 以 Food Delivery 系統的 4 個關鍵架構決策演練 ATAM 權衡點：

| 外送平台架構決策點 | 相互拉扯的品質屬性 (ATAM Trade-off Point) | 架構師的 ATAM 折衷解方 (Resolution) |
| --- | --- | --- |
| **1. 外送員 GPS 回報頻率**<br>(`每 1 秒` vs. `每 15 秒`) | **地圖即時精準度 (易用性) ↔ 手機耗電與伺服器頻寬 (資源效率)** | **動態自適應頻率**：接近顧客送達點時改為 `2 秒`；在餐廳等餐靜止時降為 `15 秒` |
| **2. 結帳扣款與廚房派單**<br>(`同步 2PC 鎖定` vs. `非同步 Saga`) | **跨服務強一致性 ↔ 暴雨尖峰高可用性與低延遲 (CAP 定理)** | **混合式 Saga**：金流授權採同步快速預扣，餐廳接單與派單則走 Kafka 非同步事件 |
| **3. AI 派單儲存庫狀態**<br>(`單一中央庫` vs. `地理分片`) | **全城全域路徑最佳化 ↔ 中央儲存庫單點故障風險 (SPOF)** | **依城市行政區分片 (Geo-Sharding)**：以 Redis 記憶體叢集分區儲存並配置熱備援 |
| **4. 優惠券與盜刷防詐檢查**<br>(`深層 ML 模型` vs. `輕量規則`) | **資安防詐嚴密性 (Security) ↔ 結帳吞吐量與低延遲 (<200ms)** | **兩階段管線**：API Gateway 內嵌 `<15ms` 輕量規則攔截；事後背景非同步跑 ML 風控 |

# 巨觀結構塑模 (1)：UML 套件圖 (Package Diagram)

## UML Package Diagram Notation Legend

### 1. Package Symbol (Tabbed Folder)




Groups logically cohesive classes, interfaces & sub-packages into a single namespace boundary.

**Name in top tab or body center.**

### 2. Element Visibility Specifiers

`+ Public Element`
Accessible by external packages

`- Private Element`
Hidden internal implementation

### 3. Dependency Arrow & Stereotypes




- Dashed open arrow: changes in Supplier may break Client.
- *«import»*: adds supplier public names to client namespace.
- *«access»*: private import (does not re-export elements).

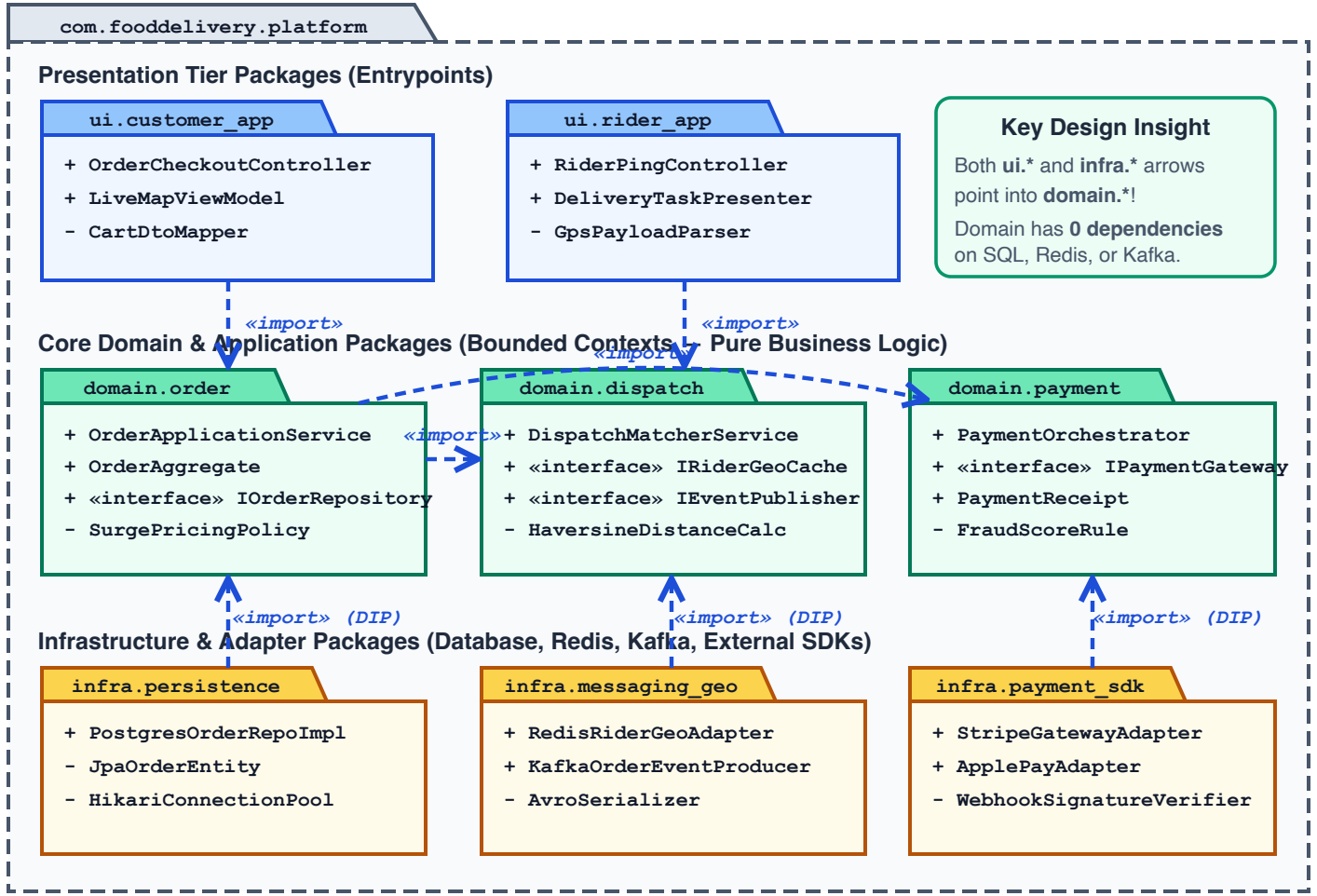
### 4. Acyclic Dependencies Principle (ADP)

- Package dependency graph must be a **Directed Acyclic Graph**.
- **UI & Infrastructure** depend on **Domain** (never reverse!).




No Cycles!

## Food Delivery System — UML Package Diagram (Module & Dependency Structure)



# UML 套件圖：核心符號規範與外送平台設計洞察

> *架構意義（編譯期模組視圖）：塑模原始碼組織結構、子系統命名空間可見度與無環模組相依規則。*

## UML 核心符號與語法規則 (Notations)

- **資料夾符號 (`Package`)**：將高內聚類別與介面歸入單一命名空間（如 `domain.order`）。
- **可見度修飾詞 (`+` Public vs. `-` Private)**：`+OrderApplicationService` 與 `+IOrderRepository` 對外公開；`-SurgePricingPolicy` 則嚴格隱藏於套件內。
- **相依虛線箭頭 (`- - ->`)**：標註 `«import»` 或 `«access»`，由相依端指向被相依端（Supplier 變動可能影響 Client）。
- **無環相依原則 (ADP)**：套件相依圖必須為有向無環圖 (DAG)，嚴禁雙向循環引用。

## Food Delivery 外送平台設計洞察

- **領域限界上下文切分**：核心業務明確拆分為 `domain.order`、`domain.dispatch` 與 `domain.payment` 三大獨立套件。
- **依賴反轉原則 (DIP) 的視覺化體現**：
  - 上層 `ui.customer_app` 向下 `«import»` `domain.order`。
  - 底層 `infra.persistence`（`PostgresOrderRepoImpl`）則是**向上反轉** `«import»` `domain.order` 中的 `IOrderRepository` 介面！
- **架構效益**：整個 `domain.*` 核心層對 PostgreSQL、Redis、Kafka 與 Stripe SDK **零編譯期相依**。

# 同儕討論：西洋棋遊戲之 MVC 架構與 UML 套件圖 (PlantUML 實戰)

## 動手塑模挑戰：以 PlantUML 繪製西洋棋 (Chess) MVC 套件圖 (4 分鐘)

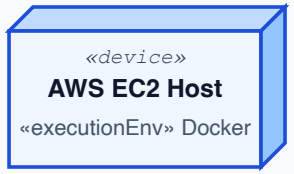
- **情境設定：** 你的團隊正採用 **MVC (Model-View-Controller)** 架構模式開發一款**西洋棋遊戲 (Chess Game)**。
- **1. 將類別歸入三大高內聚套件 (`package` 語法)：**
  - `chess.model`：`+Board`、`+Piece`、`+MoveValidator`、`-BitboardEngine`（純粹棋規與狀態，**嚴禁依賴 UI**）。
  - `chess.view`：`+BoardCanvasUI`、`+MoveHistoryPanel`、`+SoundPlayer`（畫面渲染與音效呈現）。
  - `chess.controller`：`+GameController`、`+TurnTimerHandler`（接收滑鼠點擊並調度棋局更新）。
- **2. 於 PlantText 撰寫 PlantUML 套件圖代碼：**
  - 使用 `package "chess.model" { ... }` 定義邊界，並以虛線箭頭 `..> : «import»` 連接套件。
  - **架構審查問題：** 相依箭頭 `..>` 應該指向哪個方向？`chess.model` 是否完全零對外相依？你的套件圖是否符合 **無環相依原則 (ADP)**？



# 巨觀硬體拓撲塑模 (2)：UML 部屬圖 (Deployment Diagram)

## UML Deployment Diagram Notation Legend

### 1. Node (3D Cube: Hardware or Runtime Env)



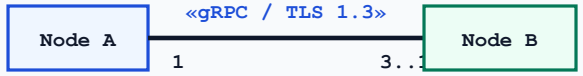

- **«device»**: Physical compute resource (Server, Phone, IoT).
- **«executionEnvironment»**: OS, Docker Container, K8s Pod, JVM, or DBMS nested inside.

### 2. Deployable Artifact («artifact»)




Concrete physical file deployed onto a Node (e.g., `.jar`, `.war`, `.ipa`, Docker image, SQL schema) that *«manifest»* a component.

### 3. Communication Path & Multiplicity

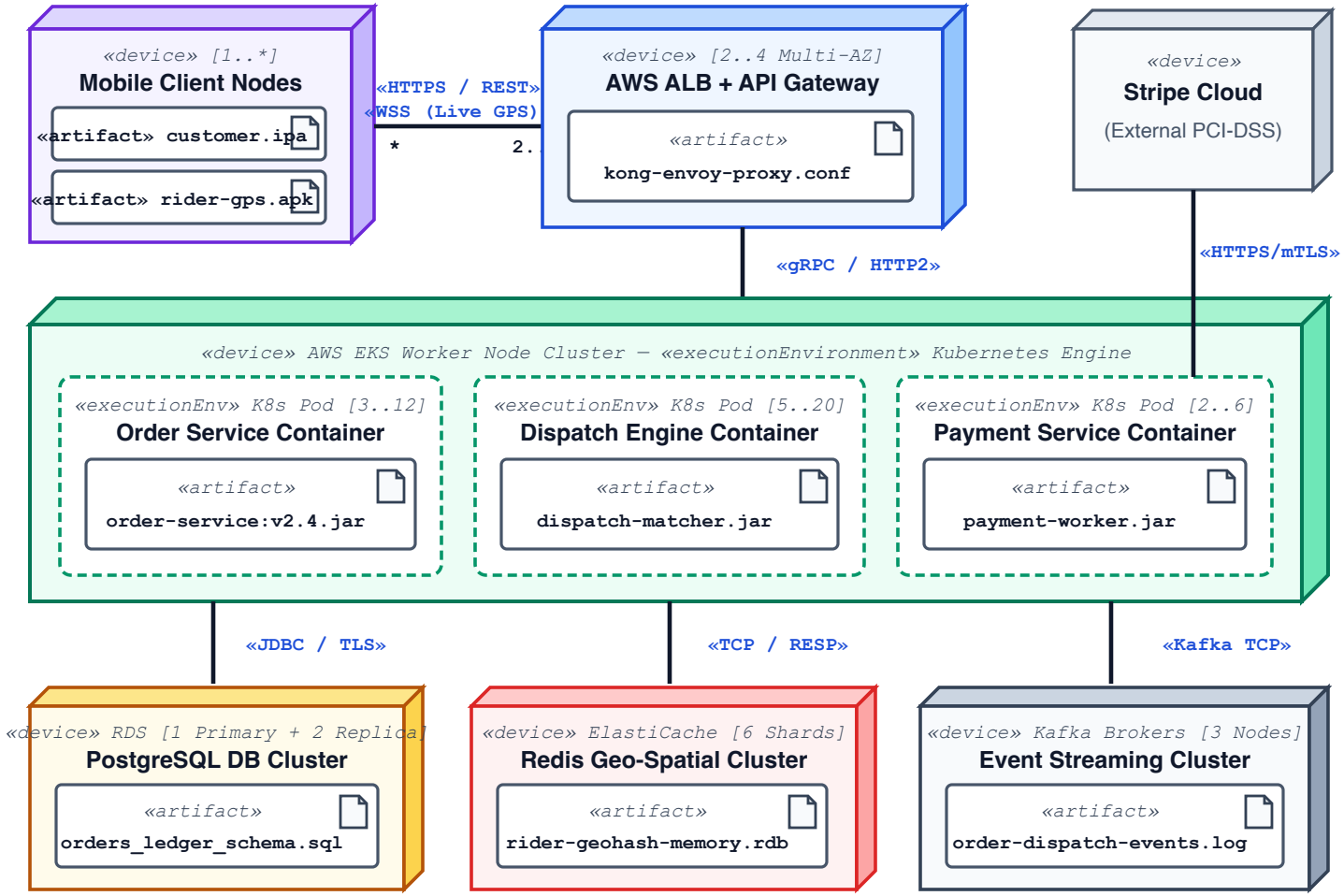



- Solid line = network link + transport protocol & cardinality.

### 4. Why Deployment Diagrams Are Vital

- Bridges **software binaries** to **cloud infrastructure**.
- Reveals **Single Points of Failure (SPOF)**, firewall boundaries, replication topology & auto-scaling pools that are 100% invisible in Class or Package diagrams!

## Food Delivery System — UML Deployment Diagram (Cloud Infrastructure)



# UML 部屬圖：3D 節點、產物與外送平台雲端高可用性解析

> *架構意義（執行期基礎設施視圖）： 塑模硬體伺服器、雲端容器叢集、實體二進位檔與網路傳輸協定。*

## UML 核心符號與語法規則 (Notations)

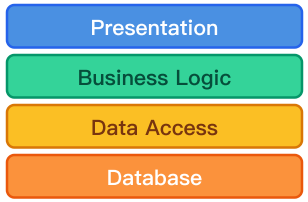
- **3D 立體節點 (`«device»` / `«executionEnv»`)**：
  - `«device»`：實體硬體或雲端主機（手機、AWS EC2、ALB）。
  - `«executionEnvironment»`：巢狀運行容器（K8s Pod、Docker、JVM、DBMS）。
- **部屬產物 (`«artifact»` + 📄 圖示)**：實體部署檔案（如 `order-service:v2.4.jar`、`rider-gps.apk`），用以實體化 (`«manifest»`) 邏輯組件。
- **通訊路徑與基數 (Multiplicity)**：實線標註底層協定（`«gRPC / HTTP2»`、`«JDBC / TLS»`）與叢集實例數（`[5..20]`）。

## Food Delivery 外送平台設計洞察

- **揭露彈性擴縮容策略 (Auto-Scaling)**：
  - 面對晚餐尖峰每秒數萬筆 GPS 座標，`Dispatch Pod [5..20]` 的擴縮上限顯著高於 `Payment Pod [2..6]`。
- **消除單點故障 (SPOF) 與高可用配置**：
  - 明確標示 `RDS [1 Primary + 2 Replica]`、`Redis [6 Shards]` 與 `Kafka [3 Nodes]` 叢集拓撲。
- **網路安全與協定邊界透明化**：
  - 外網採用 `«HTTPS / WSS»`、內網東西向採用高效能 `«gRPC»`，連往外部 `Stripe Cloud` 則強制 `«HTTPS / mTLS»`。

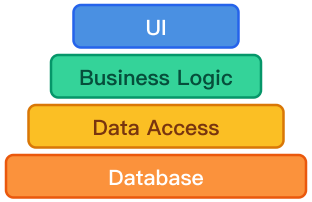
# 5.3 架構設計總結回顧：結構型 vs. 分散與資料流模式

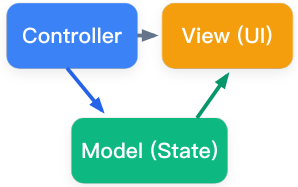
## Centralized & Structural Patterns




### 2. Layered (N-Tier) Architecture

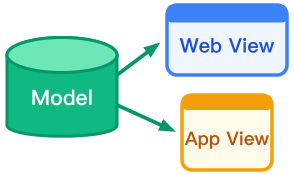
(Strict Downward Calls)

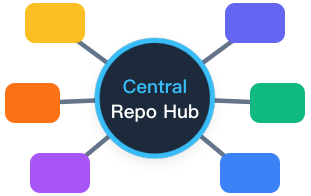



### 1. Model-View-Controller (MVC) Architecture

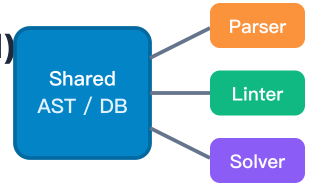
(Decoupled UI & State)

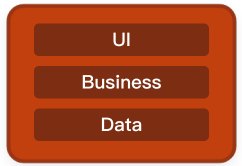


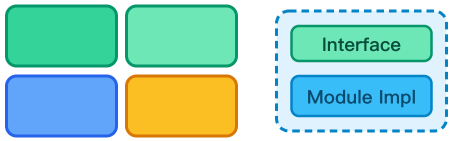

### 3. Repository (Blackboard) Architecture

(Central Shared Data Store)



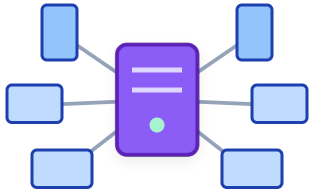




Monolithic Architecture



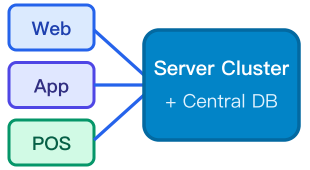

Modular Component Design

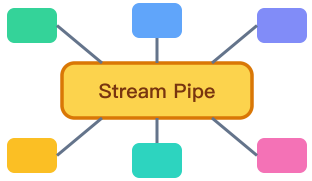
## Distributed, Routing & Flow Patterns



### 4. Client-Server Architecture

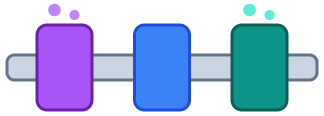
(Edge Clients ↔ Central Tier)

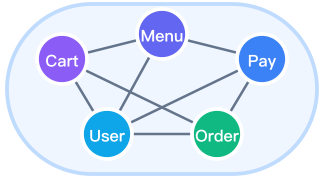



### 5. Pipe-and-Filter Stream Architecture

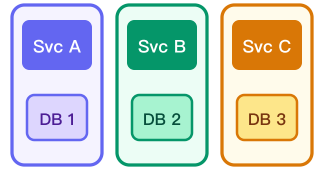
(Sequential Stateless Filters)

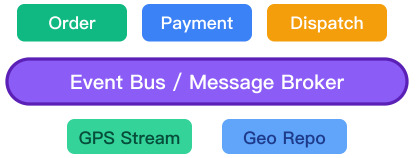



### 6. Microservices (MSA) Architecture

(Database-per-Service Pods)




Event–Driven Bus Topology

Real–World System Composition

Client–Server + MVC (Frontend)
+ Microservices + Layered (Services)
+ Pipe–Filter + Repository (Dispatch)

Multi–Pattern Hybrid Architecture

# 觀念檢核測驗 2 (CCQ 2)

在將單體架構 (Monolithic) 與微服務架構 (Microservices) 進行對比時，微服務之所以能夠提供卓越的**故障隔離能力 (Fault Isolation / 縮小衝擊半徑)**，其最核心的結構性原因為何？

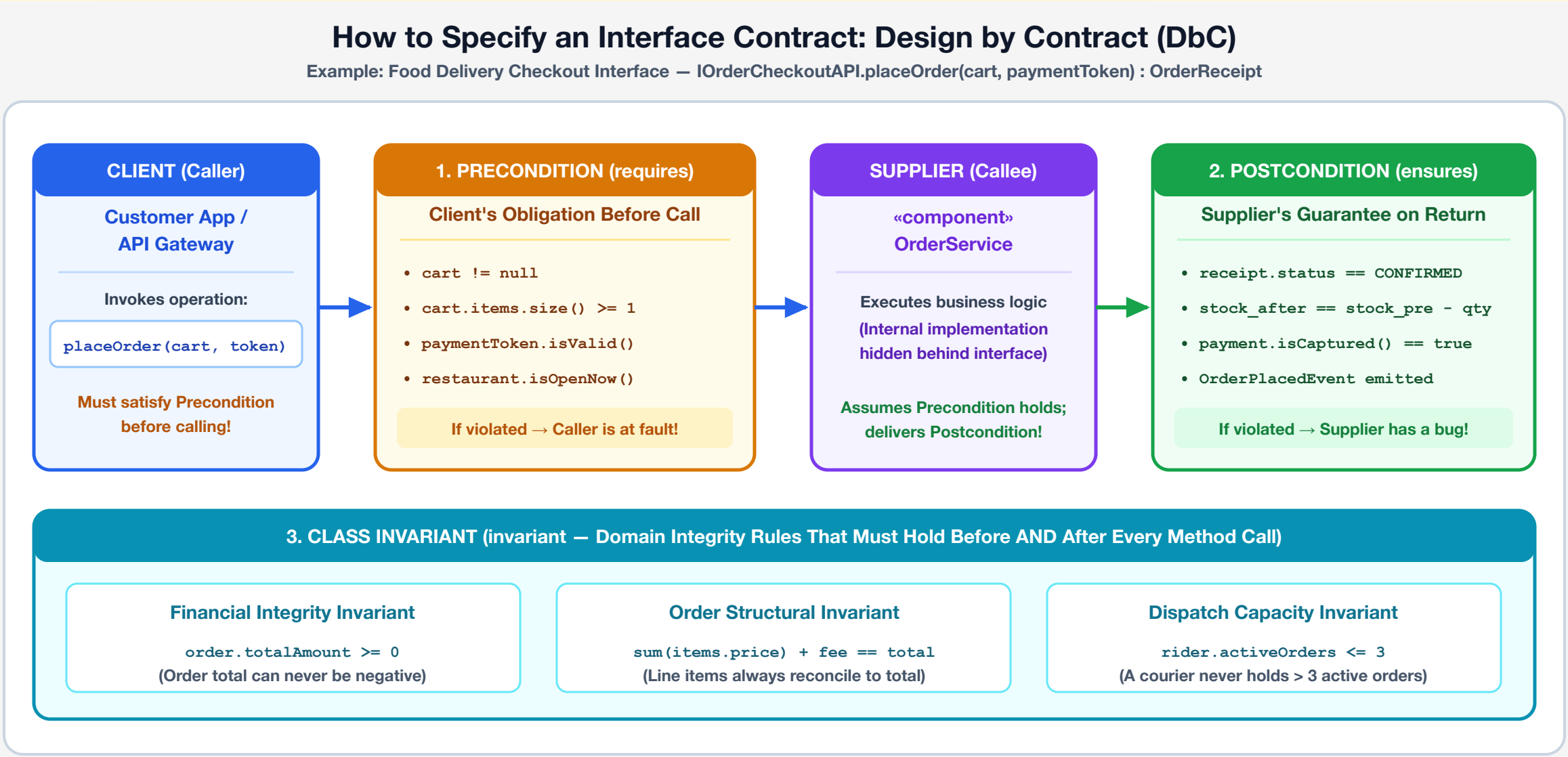
- **A.** 所有微服務統一讀寫單一共享的企業級關聯式資料庫並共用全域資料表鎖定。
- **B.** 每個微服務皆運行於獨立行程與私有資料庫中，能阻斷跨服務的骨牌式崩潰。
- **C.** 分散式微服務呼叫能完全消除跨網路封包序列化開銷與節點間的傳輸延遲。
- **D.** 將單體系統拆解為微服務後，能大幅減少雲端叢集監控與自動化維運的人力。



# 5.4 階段二 — 介面與契約規格：契約式設計 (DbC) 與組件圖

> *「將每一個軟體模組邊界視為具備明確權利 (Rights) 與義務 (Obligations) 的正式商業契約。」*
> *— Bertrand Meyer (契約式設計 Design by Contract 創立者)*

# 正式介面規格與契約式設計 (Design by Contract, DbC)



# 契約式設計 (DbC)：權利義務矩陣與工業級 API 規格標準

> ***Bertrand Meyer** 契約核心精神：* ***只要呼叫端 (Client) 履行了前置條件，供應端 (Supplier) 就有義務兌現後置條件並維持類別不變量。***

## DbC 三大契約支柱與權利義務對稱

- **1. 前置條件 (`Precondition` — 呼叫端義務)**：
  - 呼叫前必須為真（`!cart.isEmpty() && token.isValid()`）。
  - *供應端獲得之權益*：內部邏輯不必塞滿重複的防禦性檢查；若呼叫端違約，立即快速失敗 (Fail-Fast)！
- **2. 後置條件 (`Postcondition` — 供應端保證)**：
  - 執行後保證成立（`status == CONFIRMED && stock_after == stock_before - qty`）。
  - *呼叫端獲得之權益*：不必窺探供應端內部實作即可信賴結果。
- **3. 類別不變量 (`Class Invariant` — 領域恆真)**：
  - 方法執行前後恆成立（`order.totalAmount >= 0`）。

## 工業級機器可讀介面規格標準 (API Specs)

- **為什麼要在 SDD 鎖定機器可讀契約？**
  - 讓手機 App、後端微服務與 QA 能透過 Mock Server **同步平行開發與執行契約測試 (Contract Testing)**。
- **三大現代介面規格標準 (外送平台實例)**：
  - **REST API**：**OpenAPI 3.1 (Swagger)** — 明確定義端點、JSON Schema、狀態碼 (`201`, `400`, `422`) 與欄位上下界。
  - **東西向 RPC**：**Protocol Buffers (`.proto`)** — 為微服務間 `gRPC` 通訊提供強型別二進位合約。
  - **事件串流**：**AsyncAPI / Apache Avro** — 定義 Kafka 事件綱要與**冪等鍵 (`idempotency_key: UUID`)**，防止網路重試重複扣款！

# 同儕討論：撰寫氣泡排序法 `bubbleSort(int[] arr)` 之正式設計規格 (DbC)

## 雙人思辨挑戰：為 Bubble Sort 制定滴水不漏的 DbC 契約規格 (4 分鐘)

- **情境挑戰：** 寫出 `void bubbleSort(int[] arr)` 的程式碼很容易，但要寫出**嚴謹無漏洞的設計規格 (Design Spec / DbC)** 卻暗藏陷阱！請與夥伴討論以下三大要素：
- **1. 前置條件 (`Precondition / requires` — 呼叫端義務)：**
  - 呼叫前 `arr` 必須滿足什麼條件？（`arr != null`？若傳入空陣列 `arr.length == 0` 是否合法？）
- **2. 後置條件 (`Postcondition / ensures` — 供應端保證)：**
  - *經典規格漏洞：* 若後置條件只寫「執行後滿足遞增順序 `arr[0] <= arr[1] <= ... <= arr[n-1]`」夠嗎？
  - *破綻追擊：* 如果某個惡意實作直接把陣列全部填滿 `0`（變成 `[0, 0, ..., 0]`），它完全符合 `arr[i] <= arr[i+1]`！請問我們漏掉了哪一條**關鍵的第二後置條件**？（提示：多重集置換不變性 Permutation！）
- **3. 迴圈不變量 (`Loop Invariant`)：** 外層迴圈第 `k` 輪結束時，陣列最後 `k` 個元素滿足什麼性質？



# 視覺化介面契約塑模：UML 組件圖 (Component Diagram)

## UML Component Diagram Notation Legend

### 1. Component Classifier («component»)

«component»
**OrderService**

A modular, replaceable unit that encapsulates state & behavior behind explicit interfaces.
**Icon: 2-tabbed box in top-right.**

### 2. Provided Interface — "Lollipop / Ball" (—○)

Provider —○ IPaymentAPI

Contract that the component **realizes & exposes** to clients (e.g., REST/gRPC/Service API).

### 3. Required Interface — "Socket" (—( )

Consumer —( IPaymentAPI

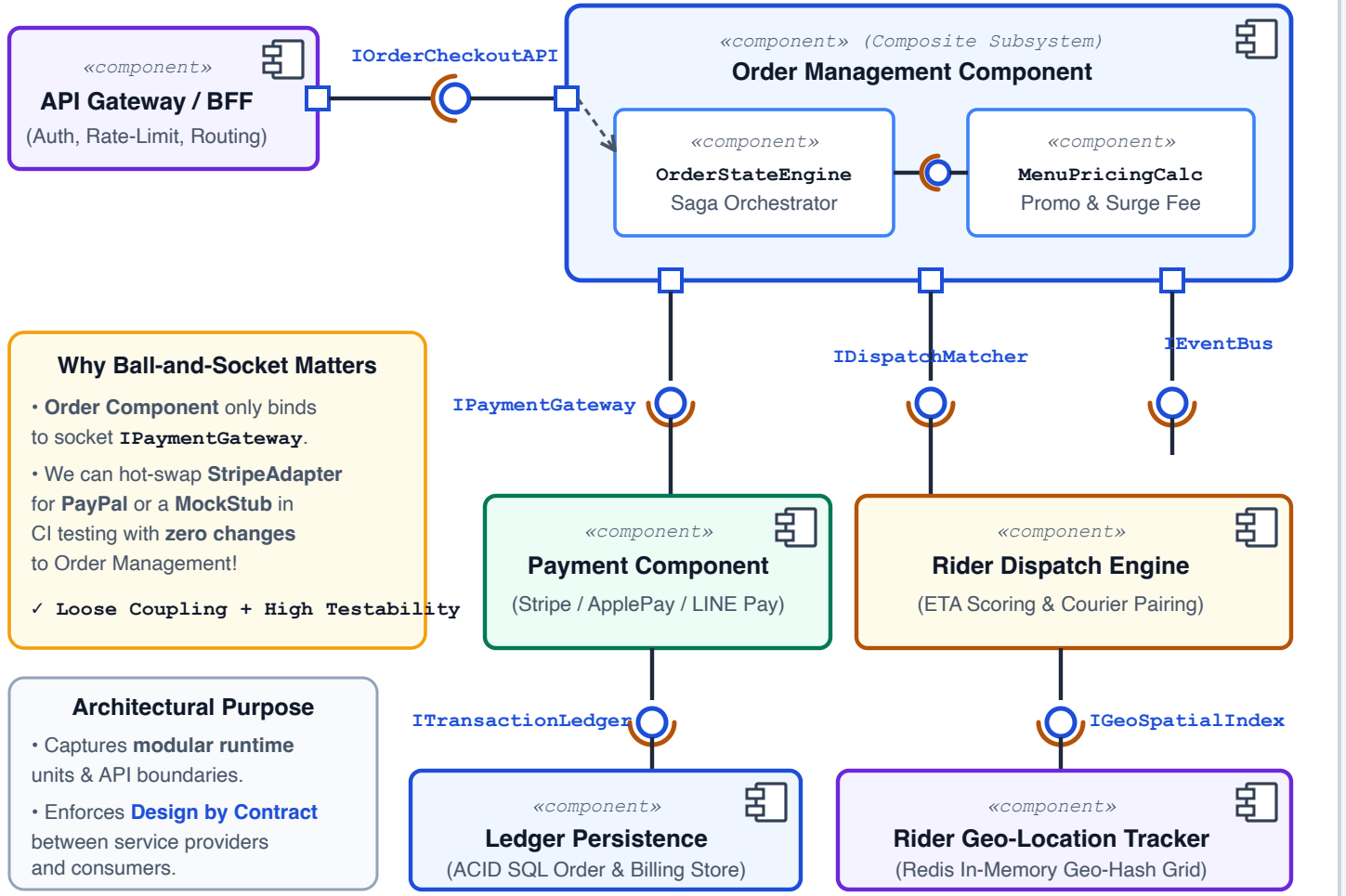
Contract that the component **requires / depends on** from an external provider to operate.

### 4. Assembly Connector (Ball-and-Socket) & Port (□)

Order —○ IPayment —( StripePay  Plug & Play!

- **Assembly Connector** —(○—: Wires consumer socket to provider lollipop. Consumer never knows provider's concrete class!
- **Port (□ on edge)**: Explicit boundary endpoint isolating external traffic & delegating (`-->`) to internal parts.

## Food Delivery System — UML Component Diagram (Interface Contracts)



# UML 組件圖：棒棒糖/插座符號與外送平台隨插即用設計

> *階段二塑模定位：塑模高階可替換軟體組件與隨插即用介面合約（將 **DbC** 契約式設計直接視覺化）。*

## UML 核心符號與 DbC 契約對應

- **組件方框 (`«component»` + ⧉ 圖示)**：封裝內部實作、僅透過明確介面與外界互動的可替換模組單元。
- **提供介面 (`–○` Lollipop 棒棒糖)**：供應端組件實作並對外提供的服務合約（承諾兌現 DbC **後置條件**）。
- **需求介面 (`–(` Socket 插座)**：呼叫端組件運作時所依賴的外部合約（承諾滿足 DbC **前置條件**）。
- **組裝連接器 (`–(○–` Ball-and-Socket)**：將需求插座扣合至提供棒棒糖，達成零實作耦合。
- **連接埠 (`□` Port) 與委派箭頭 (`- - ->`)**：組件邊界上的通訊端點，將外部請求轉發給內部子組件。

## Food Delivery 外送平台設計洞察

- **複合組件封裝 (Composite Component)**：`Order Management` 內部封裝了 `OrderStateEngine` 與 `MenuPricingCalc`，外界僅能透過邊界 Port (`□`) 存取。
- **熱插拔替換能力 (Plug-and-Play)**：
  - `Order Management` 只認得 `IPaymentGateway` 插座。
  - 無論底層接上的是 `Stripe`、`ApplePay`，還是自動化測試時的 `MockPaymentStub` 棒棒糖，訂單組件皆**零程式碼修改**！
- **派單與空間索引解耦**：`Rider Dispatch Engine` 僅透過 `IGeoSpatialIndex` 介面查詢外送員座標。

# 觀念檢核測驗 3 (CCQ 3)

在設計流程的第二階段（**介面與契約規格**）中，當外送平台的 `Order Management` 組件透過 `IPaymentGateway` 介面呼叫 `Payment Component` 時，**契約式設計 (DbC)** 與 **UML 組件圖** 是如何對應運作的？

- **A.** `Order Management` 對外伸出棒棒糖 (`—○`)，且可任意傳入負數金額而不必負責驗證。
- **B.** 呼叫端插座 (`—(`) 負責滿足**前置條件**，而供應端棒棒糖 (`—○`) 保證兌現**後置條件**。
- **C.** UML 組件圖透過 3D 硬體節點方塊 (`«device»`) 自動略過所有執行期前後置條件檢查。
- **D.** 契約式設計要求呼叫端組件直接跨越邊界存取並修改供應端內部的私有資料表。

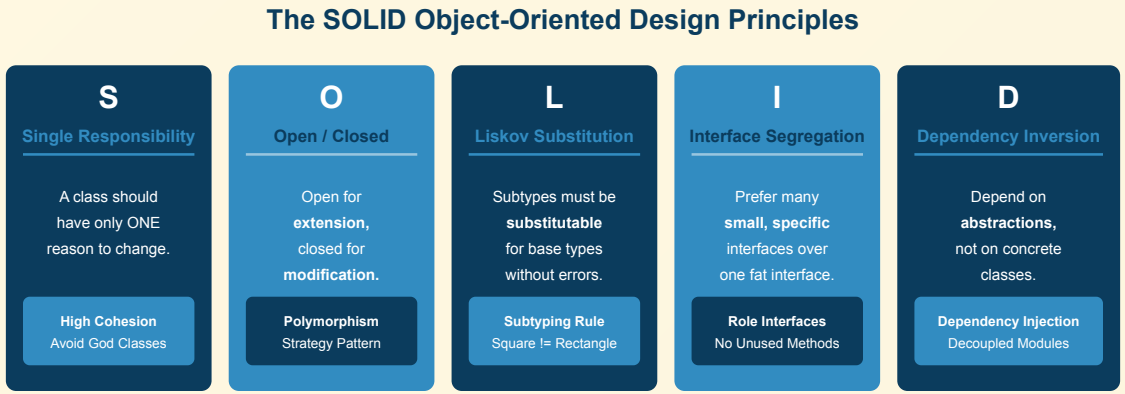
# 5.5 階段三 — 組件與物件導向設計：設計模式、SOLID 與類別圖

> *"優良的程式碼總是看起來像是出自一位真正關心它的人之手。"*
> *— Robert C. Martin (Uncle Bob)*

# 什麼是設計模式 (Design Patterns)？從 GoF 到 SOLID 基石

- **什麼是軟體設計模式 (Software Design Pattern)？**
  - **架構模式 (Architectural Patterns, 5.3 節)** 定義全系統的巨觀拓撲；而 **設計模式 (Design Patterns)** 則是針對單一組件內部「反覆出現的類別與物件設計問題」，提供可重用、經實務驗證的微觀結構解法。
- **百花齊放的設計模式家族（以經典 GoF 四人幫 23 種模式為例）：**
  - **生成型模式 (Creational, 5 種)**：*Factory Method, Abstract Factory, Builder, Singleton, Prototype*（解耦物件的建立過程）。
  - **結構型模式 (Structural, 7 種)**：*Adapter, Decorator, Facade, Proxy, Composite, Bridge*（優雅組合類別與介面）。
  - **行為型模式 (Behavioral, 11 種)**：*Strategy, Observer, Command, State, Template Method, Iterator*（協調物件間的職責與互動）。
- **本章聚焦核心 —— SOLID 五大設計原則 (Robert C. Martin)：**
  - 與其死背 23 種 GoF 模式，我們在此聚焦於支撐所有設計模式的**「第一性原理 (First Principles)」—— SOLID**（例如：*Strategy* 體現 **OCP**、*Adapter* 體現 **DIP**）！

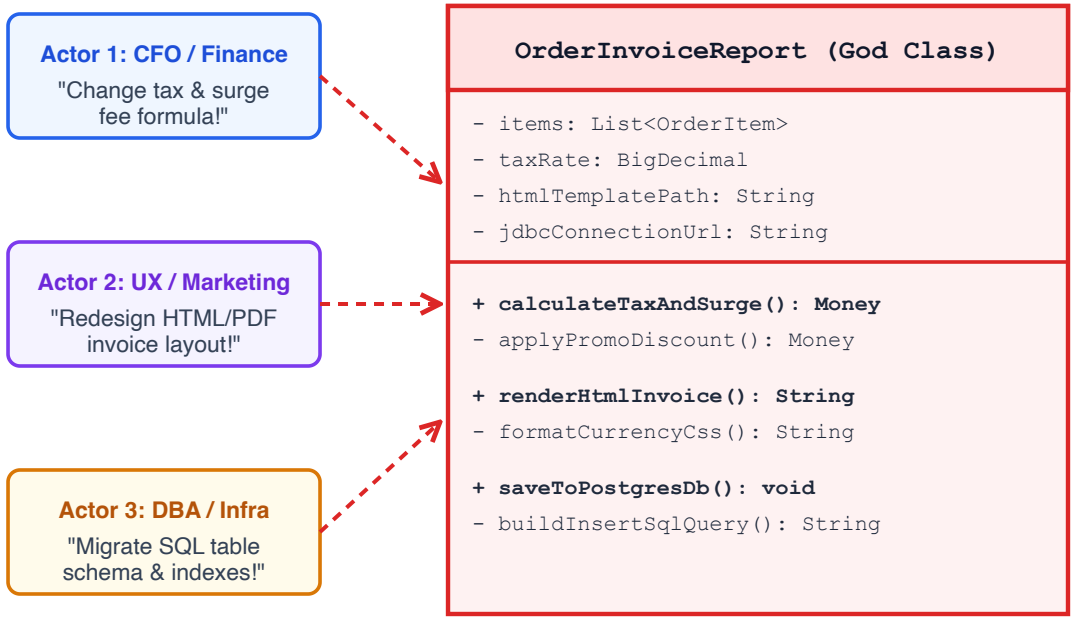
# 概覽：以 UML 結構視角掌握 SOLID 五大設計原則




- **SOLID 如何決定 UML 類別與介面的拓撲結構：**
  - **S — 單一職責原則 (SRP)**：拆解臃腫的「上帝類別」，確保每個類別方框僅服務**單一利益關係人 (1 Actor / 1 Reason to Change)**。
  - **O — 開放封閉原則 (OCP)**：以穩定的 `«interface»` **抽象介面**取代硬編碼相依，透過新增實作類別 (`- - -|>`) 無痛擴充。
  - **L — 里氏替換原則 (LSP)**：確保繼承 (`——|>`) 與實作 (`- - -|>`) 關係嚴格遵守父型態承諾之**行為契約與後置條件**。
  - **I — 介面隔離原則 (ISP)**：將包山包海的「肥大介面」拆解為多個**精巧專一的角色介面 (`Role Interfaces`)**。
  - **D — 相依反轉原則 (DIP)**：將跨層相依箭頭**向上反轉**，使高階業務與低階實作共同依賴於領域抽象介面。

# S — 單一職責原則 (SRP)：UML 重構對照

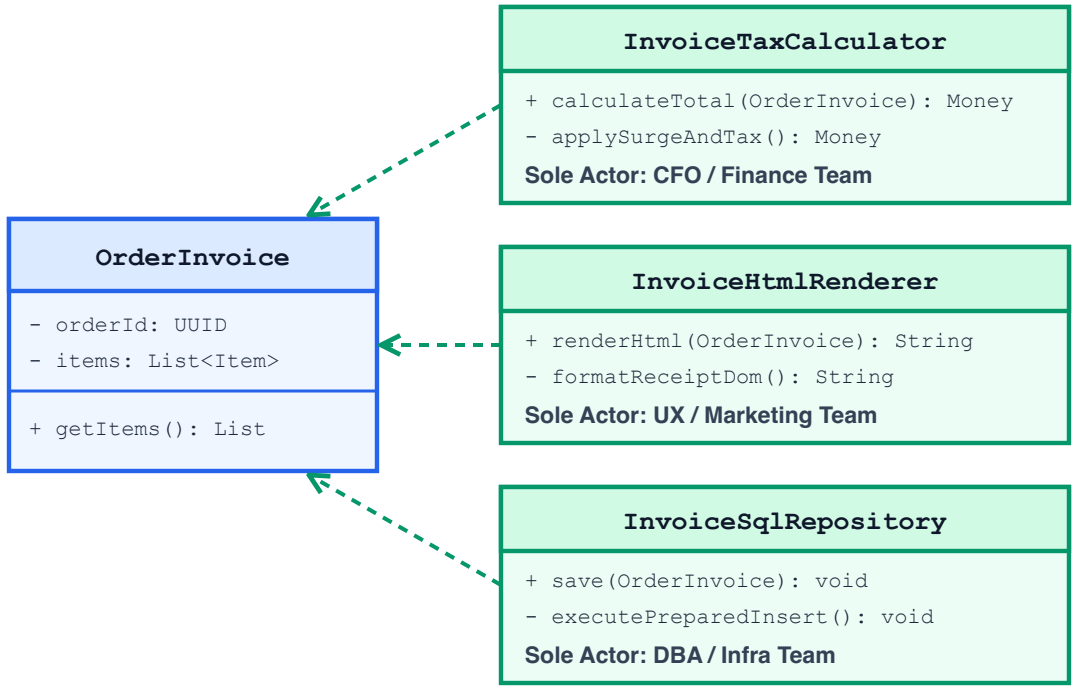
## ✘ Violating SRP: God Class with 3 Reasons to Change




**Why This UML Design Fails (Coincidental / Logical Cohesion):**

- **3 Orthogonal Axes of Change** collide inside one class file.
- A routine UI tweak by Marketing (`renderHtmlInvoice`) forces recompiling and redeploying mission-critical CFO tax calculation & SQL persistence!
- Frequent Git merge conflicts across Finance, Frontend, and Backend teams.

## ✓ Following SRP: 1 Cohesive Class per Actor




**Why This UML Design Excels (Functional Cohesion + Data Coupling):**

- **1 Class = 1 Stakeholder = 1 Reason to Change**.
- UI changes in `InvoiceHtmlRenderer` have **zero blast radius** on tax calculations or SQL persistence; each class can be unit-tested in isolation!

# S — 單一職責原則 (SRP)：UML 結構與設計解析

> *核心定義 (Robert C. Martin)：「一個類別應該有且僅有一個引起它改變的原因（亦即：僅對單一利益關係人 Actor 負責）。」*

## ❌ UML 反模式：多重角色的「上帝類別」

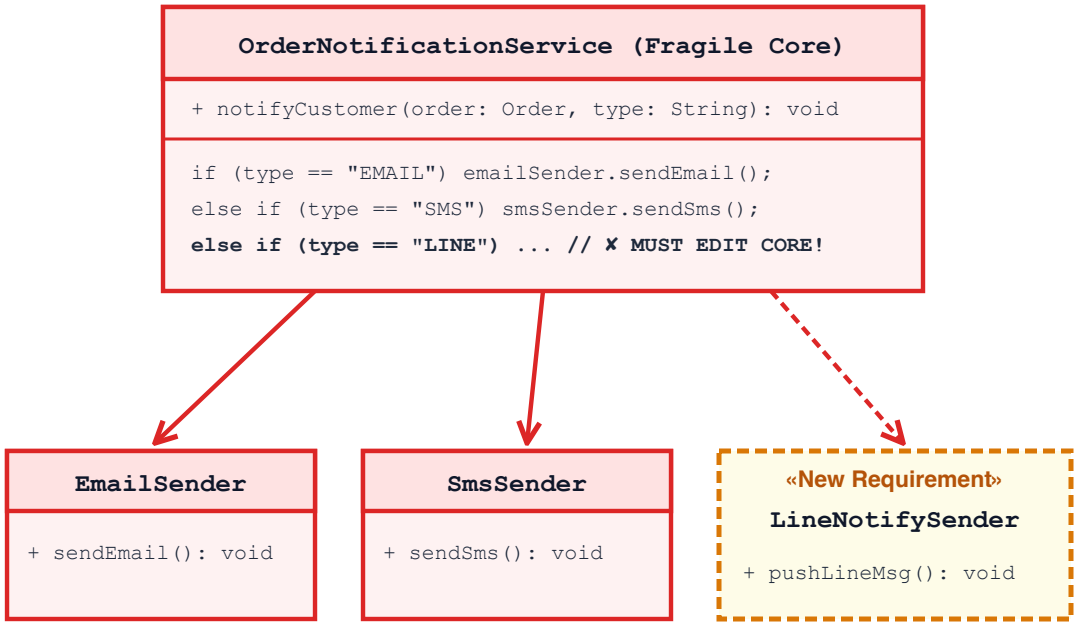
- **三條正交變更軸線碰撞於同一類別**：
  - `OrderInvoiceReport` 同時混雜了**財務計稅邏輯** (`calculateTaxAndSurge`)、**網頁排版渲染** (`renderHtmlInvoice`) 與 **SQL 資料庫寫入** (`saveToPostgresDb`)。
- **高脆弱性與 Git 合併衝突**：
  - 行銷部僅要求微調發票 HTML 配色，卻迫使包含核心計稅公式的整個類別重新編譯部署，極易引發非預期迴歸錯誤（淪為低階之**邏輯/巧合內聚**）。

## ✅ UML 重構方案：一類別一角色 (1 Class per Actor)

- **依職責邊界高內聚拆解**：
  - `InvoiceTaxCalculator` ➔ 唯一變更源：**財務部門 (CFO)**。
  - `InvoiceHtmlRenderer` ➔ 唯一變更源：**UX / 行銷部門**。
  - `InvoiceSqlRepository` ➔ 唯一變更源：**DBA / 基礎設施團隊**。
- **架構效益**：
  - 達成最高等級之 **功能內聚 (Functional Cohesion)** 與 **資料耦合 (Data Coupling，僅傳遞 `OrderInvoice`)**，三個團隊可平行開發測試、互不干擾。

# O — 開放封閉原則 (OCP)：UML 重構對照

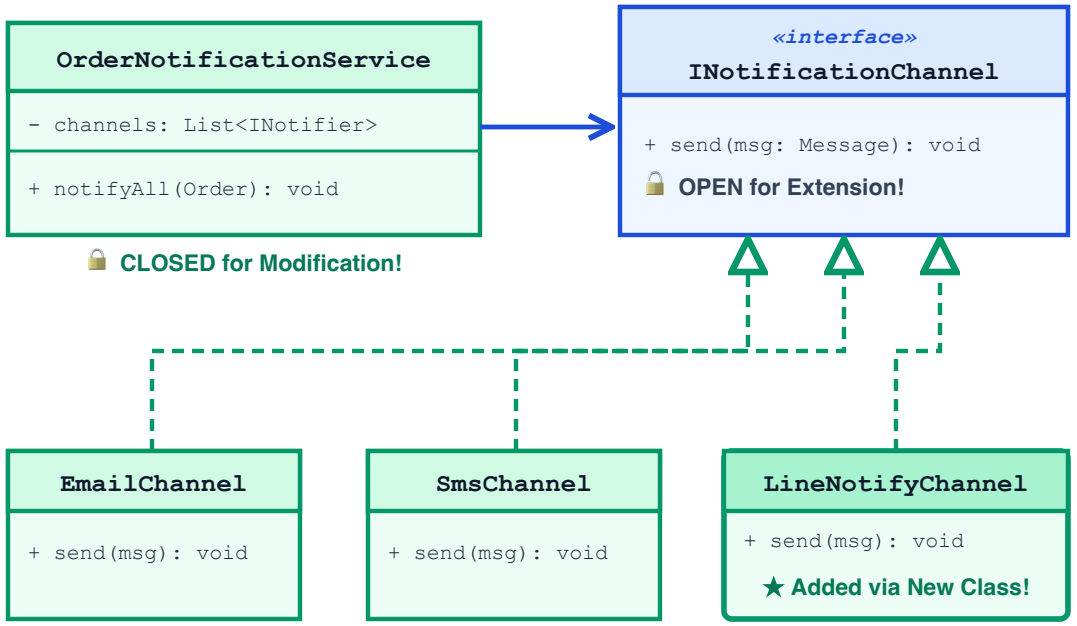
## ✘ Violating OCP: Rigid Concrete Coupling & Switch Chains




**Why This Violates OCP (Not Closed for Modification):**

- `OrderNotificationService` is directly coupled to every concrete channel.
- Every time business adds a new channel (LINE, WhatsApp, Discord), engineers must **cut open and modify** the tested core service class.
- High risk of breaking existing Email/SMS logic during routine extensions!

## ✓ Following OCP: Open for Extension, Closed for Modification




**Why This UML Design Excels (Polymorphic Strategy / Plugin Pattern):**

- `OrderNotificationService` depends solely on the stable `INotificationChannel`.
- To support **LINE Notify** or **Discord**, we simply add a new class that realizes (- - -|>) the interface—**0 lines modified** in the core service!
- Eliminates regression bugs in existing battle-tested notification channels.

# O — 開放封閉原則 (OCP)：UML 結構與設計解析

> ***核心定義 (Bertrand Meyer)：「軟體實體（類別、模組、組件）應當對擴充開放 (Open for Extension)，但對修改封閉 (Closed for Modification)。」***

## ❌ UML 反模式：直接耦合具體類別與條件分支

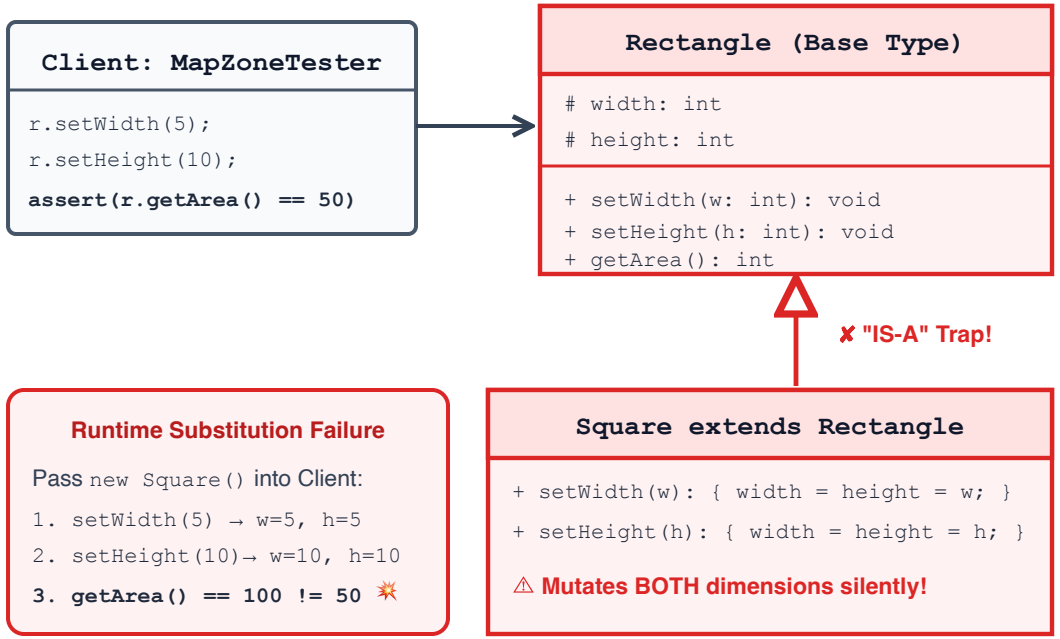
- **放射狀硬編碼相依 (`——>`)**：
  - `OrderNotificationService` 直接依賴所有具體通知類別 (`EmailSender`, `SmsSender`)，並以 `if / else if` 分派。
- **違反「對修改封閉」**：
  - 每當外送平台新增一種通知管道（如 `LineNotifySender`、`WhatsApp` 或 `Discord`），都必須**打開並修改**核心服務類別，隨時可能改壞原本正常的 Email/SMS 流程！

## ✅ UML 重構方案：多型抽象介面樞紐 (Strategy / Plugin)

- **穩定抽象契約 (`«interface» INotificationChannel`)**：
  - `OrderNotificationService` 僅依賴 `INotificationChannel.send(msg)` 介面（🔒 **對修改封閉**）。
- **無限外掛擴充 (`- - -|>`)**：
  - `EmailChannel`、`SmsChannel` 與新增之 `LineNotifyChannel` 各自獨立實作該介面（🔓 **對擴充開放**）。
- **零侵入擴展**：新增通知管道只需新增一個實作類別，核心調度服務保持 **零修改、零迴歸風險**。

# L — 里氏替換原則 (LSP)：UML 重構對照

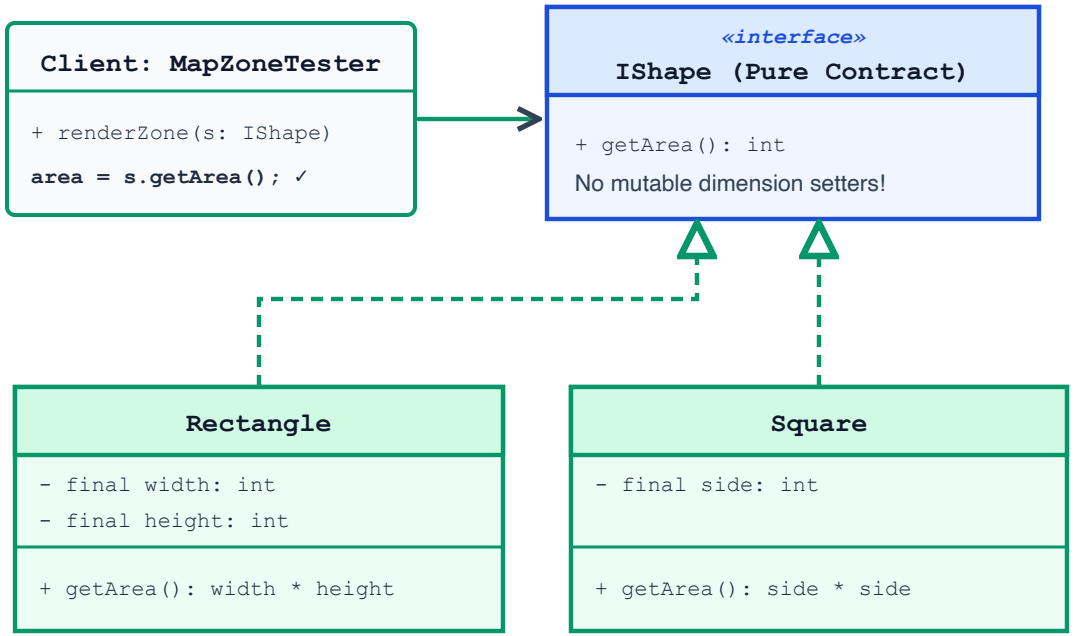
## ✘ Violating LSP: Subclass Breaks Base Class Postcondition




**Why This Violates Liskov Substitution Principle (LSP):**

- **Rectangle Postcondition Contract**: `setHeight(h)` modifies `height` while leaving `width` invariant.
- `Square` violates this postcondition! Mathematical "IS-A" does NOT imply behavioral substitutability when mutable state setters are involved.

## ✓ Following LSP: Substitutable Contract via Shared Abstraction




**Why This UML Design Excels (100% Behavioral Substitutability):**

- `Rectangle` and `Square` are independent siblings realizing `IShape`.
- Any `IShape` instance can substitute for another without hidden side effects, broken invariants, or `UnsupportedOperationException`!
- **Design by Contract Rule**: Subtypes never weaken preconditions or violate postconditions & invariants of the base contract.

# L — 里氏替換原則 (LSP)：UML 結構與設計解析

> *核心定義 (Barbara Liskov)：「子型態 (Subtypes) 必須能夠在行為上無痛替換其基礎型態 (Base Types)，而不改變程式任何預期的正確性。」*

## ❌ UML 反模式：誤踩「語意 IS-A」繼承陷阱

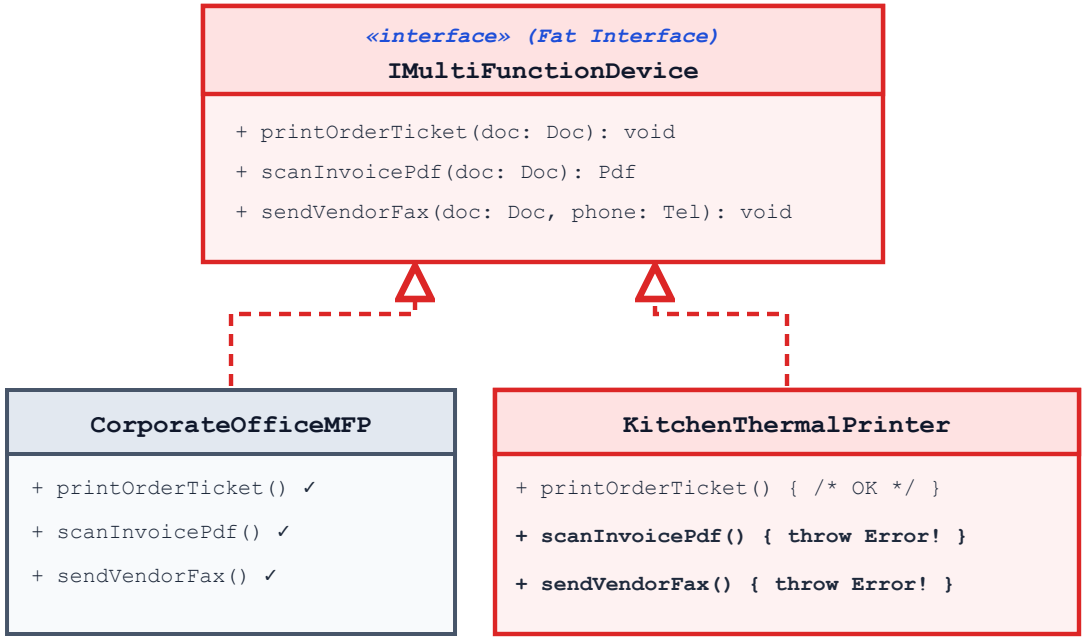
- **破壞父類別後置條件 (`Square ──|> Rectangle`)**：
  - 父類別 `Rectangle` 承諾：`setHeight(10)` 僅改變高度，`width` 維持不變。
  - 子類別 `Square` 覆寫時暗中連帶修改 `width`，導致呼叫端執行 `w=5, h=10` 時算出 `Area = 100 != 50`！
- **外送平台常見之 LSP 違規反例**：
  - 讓 `貨到付款 (CashOnDelivery)` 繼承 `線上刷卡 (OnlinePayment)`，卻在覆寫 `autoRefundCard()` 時拋出 `UnsupportedOperationException`！

## ✅ UML 重構方案：行為子型態化 (Behavioral Subtyping)

- **解除錯誤繼承，改採平輩契約實作 (`- - -|> IShape`)**：
  - 移除 `Square` 對可變 `Rectangle` 的繼承箭頭。
  - 兩者改為不可變類別，平輩實作純粹的 `«interface» IShape` (`+ getArea(): int`)。
- **契約式設計 (Design by Contract) 鐵律**：
  - 子型態**絕不可強化前置條件**、**絕不可弱化後置條件**，亦**不可拋出父型態未宣告之異常**。

# I — 介面隔離原則 (ISP)：UML 重構對照

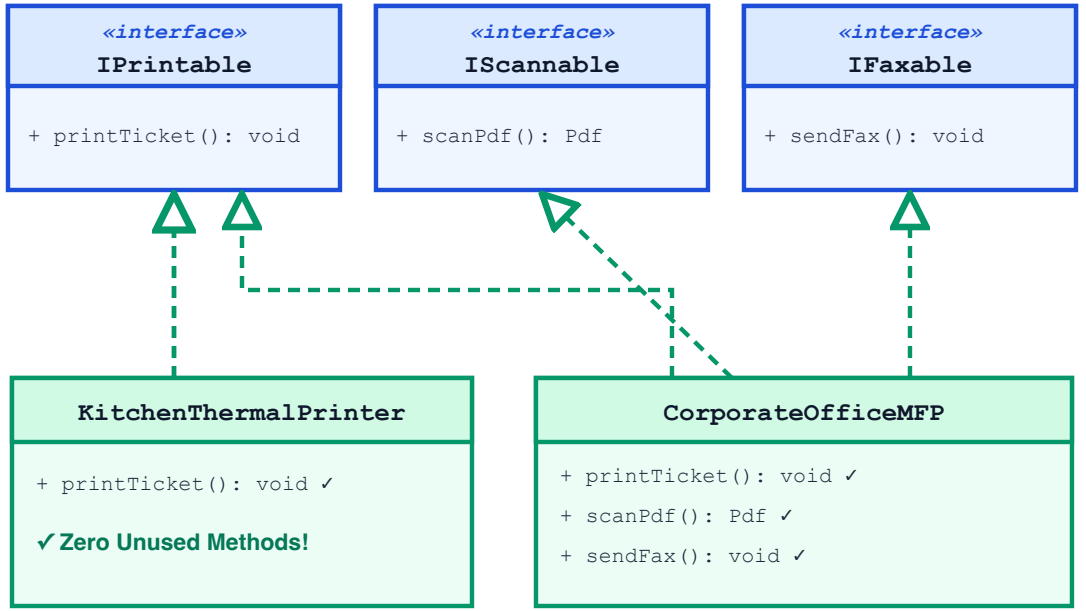
## ✘ Violating ISP: Bloated "Fat" Interface Pollutes Clients




**Why This Violates Interface Segregation Principle (ISP):**

- `KitchenThermalPrinter` only prints restaurant receipts, yet is forced to implement dummy `scan` and `fax` methods that throw runtime exceptions!
- **Phantom Coupling**: If the `Tel` parameter in `sendVendorFax()` changes, `KitchenThermalPrinter` is forced to recompile & redeploy for no reason!

## ✓ Following ISP: Lean, Role-Specific Interfaces




**Why This UML Design Excels (Cohesive Role Contracts):**

- Each interface represents a single capability (`IPrintable`, `IScannable`).
- `KitchenThermalPrinter` implements only `IPrintable`—no dummy stubs!
- Multi-function devices compose multiple interfaces via multiple realization (`implements IPrintable, IScannable, IFaxable`).

# I — 介面隔離原則 (ISP)：UML 結構與設計解析

> *核心定義 (Robert C. Martin)：「客戶端（與實作類別）不應該被迫依賴於它們所不使用的方法或介面。」*

## ❌ UML 反模式：包山包海的「肥大介面 (Fat Interface)」

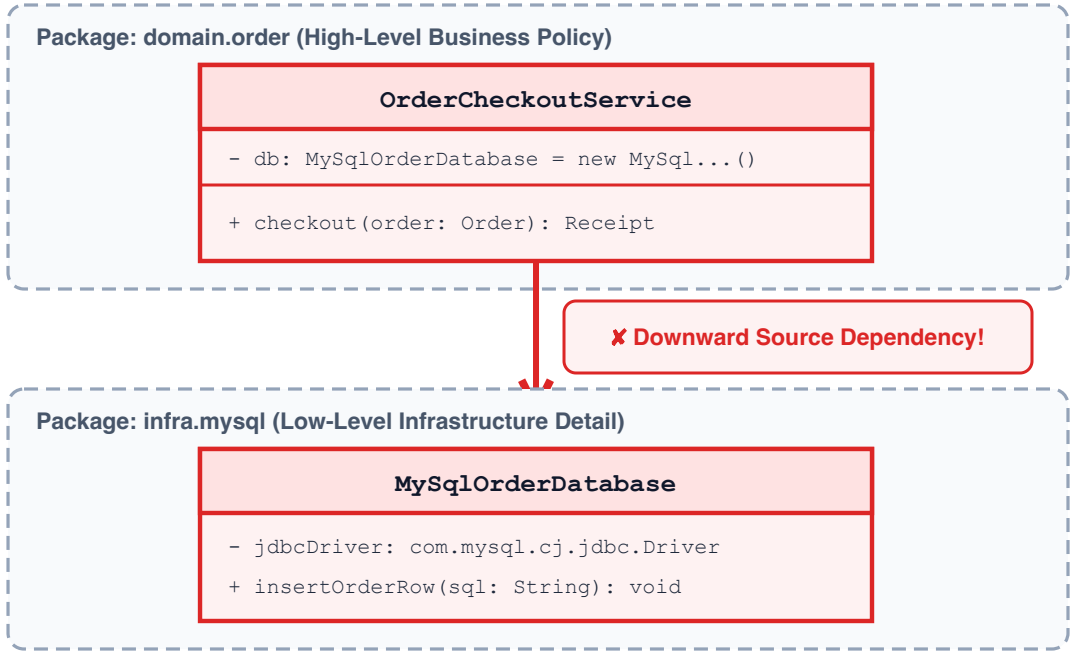
- **受污染的單一合約 (`IMultiFunctionDevice`)**：
  - 將毫無關聯的列印、掃描與傳真方法硬塞進同一個 `«interface»` 方框中。
- **引發兩大架構災難**：
  - **1. 強迫實作空殼方法 (連帶違反 LSP)**：`KitchenThermalPrinter` 被迫在 `scan()` 與 `fax()` 中拋出異常。
  - **2. 幽靈重編譯耦合 (Phantom Coupling)**：當傳真方法 `sendVendorFax()` 的參數型態修改時，完全沒有傳真功能的廚房出單機竟然被迫重新編譯！

## ✅ UML 重構方案：高內聚之角色介面 (Role Interfaces)

- **依角色能力精準切分介面**：
  - 拆解為 `«interface» IPrintable`、`«interface» IScannable` 與 `«interface» IFaxable`。
- **按需實作與多介面組合 (`- - -|>`)**：
  - 餐廳廚房出單機 `KitchenThermalPrinter` **僅實作** **`IPrintable`**。
  - 高階事務機 `CorporateOfficeMFP` 則同時實作三個角色介面。
- **架構效益**：徹底消除空殼方法與跨模組的幽靈耦合。

# D — 相依反轉原則 (DIP)：UML 重構對照

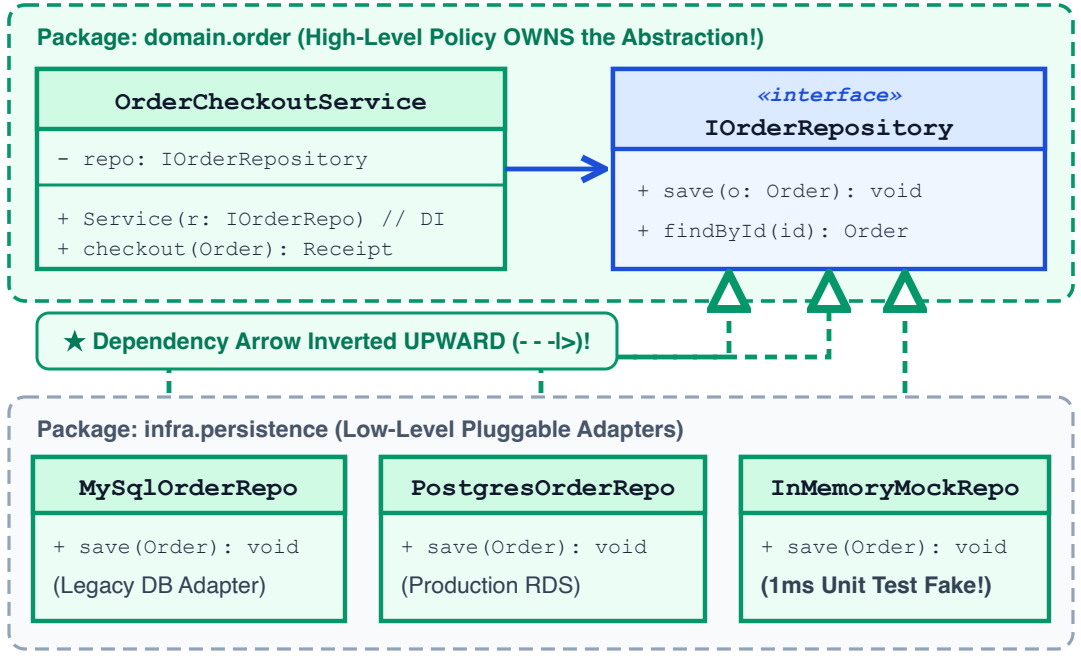
## ✘ Violating DIP: High-Level Policy Coupled to Low-Level Detail




**Why This Violates Dependency Inversion Principle (DIP):**

- **High-level business policy** directly imports low-level MySQL JDBC details.
- **Immobility**: Cannot reuse `OrderCheckoutService` with PostgreSQL or DynamoDB without rewriting core business classes.
- **Untestable**: Cannot run fast unit tests without booting a real MySQL DB!

## ✓ Following DIP: Both Layers Depend on Domain Abstraction




**Why This UML Design Excels (Inversion of Control & Clean Architecture):**

- **Rule 1**: High-level `OrderCheckoutService` and low-level `PostgresOrderRepo` both depend on the `IOrderRepository` abstraction.
- **Rule 2**: Low-level DB adapters point **UPWARD** into the Domain package; injected via Constructor Dependency Injection (DI) for 1ms unit testing!

# D — 相依反轉原則 (DIP)：UML 結構與設計解析

> ***核心定義 (Robert C. Martin)：*** *「1. 高階模組不應依賴低階模組，兩者皆應依賴於抽象介面。 2. 抽象介面不應依賴於細節實作，細節實作必須依賴於抽象介面。」*

## ❌ UML 反模式：自上而下的傳統相依 (`domain ──> infra`)

- **高階商業邏輯綁死低階細節**：
  - `OrderCheckoutService` 直接持有並引用具體的 `MySqlOrderDatabase` 類別。
- **引發兩大致命問題**：
  - **不可移植性 (Immobility)**：若要將資料庫遷移至 PostgreSQL，必須修改高階領域類別。
  - **無法獨立單元測試**：要測試 `OrderCheckoutService` 的結帳計價邏輯，竟然必須先啟動一台真實的 MySQL 伺服器！

## ✅ UML 重構方案：箭頭向上反轉 (`infra - - -|> domain`)

- **領域層擁有抽象介面所有權 (`IOrderRepository`)**：
  - `OrderCheckoutService` 僅相依於同屬 `domain.order` 套件的 `«interface» IOrderRepository`。
- **低階適配器向上實作 (`- - -|>`)**：
  - `PostgresOrderRepo` 與 `InMemoryMockRepo` 的實作箭頭**跨越套件邊界向上反轉**指向 `IOrderRepository`。
- **建構子依賴注入 (DI)**：正式環境注入 `PostgresOrderRepo`；單元測試則注入 `InMemoryMockRepo`，達成 **1 毫秒極速隔離測試**！

# 同儕討論：探索 GoF 觀察者模式 (Observer Pattern) 之設計動機與結構

## 網路檢索與雙人結構剖析：GoF 觀察者設計模式 (4 分鐘)

- **1. 動手網路查詢：** 請上網搜尋經典 GoF **Observer Design Pattern（觀察者模式）**（如 *Refactoring.Guru*）。
- **2. 探討設計動機 (Motivation — 它解決什麼痛點？)：**
  - 當單一**被觀察主題 (`Subject`)** 狀態改變時（例如外送平台 `Order` 狀態變為 `READY_FOR_PICKUP`），多個**相依觀察者**（`顧客推播`、`外送員地圖`、`餐廳看板`）必須自動收到通知並更新——且**不能在 `Order` 內部寫死對這些具體類別的直接呼叫**！
- **3. 剖析 UML 結構與 SOLID 關聯（請與夥伴討論並繪出草圖）：**
  - 找出其 UML 類別圖中的四大角色：`Subject (Publisher)`、`«interface» IObserver (Subscriber)` 與多個 `ConcreteObserver`。
  - 觀察者模式是如何同時體現 **OCP（開放封閉原則）** 與 **DIP（相依反轉原則）** 的？



# 觀念檢核測驗 4 (CCQ 4)

哪一個 SOLID 物件導向設計原則明確主張：當需要為系統引入新功能時，應當透過「新增新的類別」來擴充，而非直接修改既有已通過測試的原始碼？

- **A.** 單一職責原則 (SRP)
- **B.** 開放封閉原則 (OCP)
- **C.** 介面隔離原則 (ISP)
- **D.** 里氏替換原則 (LSP)

# 5.6 階段四 — 資料與持久化設計：ERD 塑模、快取與多模態儲存

> *「差勁的程式設計師擔心程式碼；優秀的程式設計師關心資料結構以及它們之間的關係。」*
> *— Linus Torvalds (Linux 與 Git 創立者)*

# 三層次資料塑模、UML ERD 與外送平台多模態儲存



# 1. 三層次資料塑模、UML ERD 符號與正規化 (3NF)

> *階段四塑模定位： 將領域實體轉化為具備參照完整性的關聯式綱要 (ERD) 與高速實體索引。*

## 三層次資料塑模與 UML ERD 符號

- **1. 概念模型 → 2. 邏輯 ERD → 3. 實體綱要**：
  - 由商業實體推進至正規化資料表結構，最終落實為 DBMS 專屬索引與分區策略。
- **UML 資料 / ERD 核心符號規範**：
  - `«PK»` **(Primary Key 主鍵)**：唯一識別每筆資料列之不可變鍵值（如 `order_id: UUID`）。
  - `«FK»` **(Foreign Key 外來鍵)**：維護跨表參照完整性（如 `orders.customer_id` 參照 `customers`）。
  - **基數與組合 (`1 — 0..*`, `1 ◆— 1..*`)**：區分一般關聯與生命週期綁定的強組合（`Order` ◆—— `OrderItem`）。

## 第三正規化 (3NF) vs. 實體效能最佳化

- **第三正規化 (3NF — 確保寫入一致性)**：
  - 每個非鍵欄位皆**僅直接相依於主鍵**（例如：`orders` 只存 `restaurant_id`，絕不在百萬筆訂單中重複抄寫餐廳地址字串）。
  - **效益**：徹底消除更新異常與冗餘不一致。
- **實體索引與分區調校 (加速讀取效能)**：
  - **複合索引 (`«idx»`)**：建立 `(customer_id, created_at DESC)` 複合 B-Tree 索引，使查詢「歷史訂單」由 `800ms` 全表掃描降至 `< 2ms`！
  - **按月表分區 (Partitioning)**：依 `created_at` 按月切分 `orders` 表，確保熱資料常駐記憶體。

# 2. 多模態持久化 (Polyglot Persistence)、快取與分散式一致性

> *現代架構鐵律：根據資料的一致性要求、讀寫頻率與結構特性，組合關聯式資料庫、記憶體快取與事件日誌。*

## 外送平台之多模態儲存引擎分工

- **1. 關聯式交易庫 (`PostgreSQL` — 強 ACID)**：
  - *適用場景*：訂單 (`orders`)、明細與金流 (`payments`)。
  - *關鍵設計*：在 `payments` 表強制設定唯一約束 `«UQ» idempotency_key`，從資料庫底層杜絕重複扣款。
- **2. 記憶體空間與鍵值快取 (`Redis` — 高速暫態)**：
  - *適用場景*：外送員即時座標 (`rider:geo:{city}`) 與熱門餐廳菜單 **Cache-Aside 快取** (`TTL = 5 分鐘`)。
  - *關鍵設計*：支援每秒萬次寫入與 `< 5ms` 半徑查詢；設 `TTL = 30 秒` 自動剔除斷線外送員。
- **3. 不可變事件串流日誌 (`Kafka` — 非同步解耦)**：
  - *適用場景*：以 `order_id` 為 Partition Key 確保同筆訂單狀態事件嚴格保序。

## 跨資料源一致性：ACID vs. BASE 與 Outbox

- **單一服務內部 (`ACID` 本地交易)**：
  - 藉由關聯式資料庫 Transaction 保證原子性與隔離性。
- **跨微服務與異質儲存 (`BASE` 最終一致性)**：
  - *經典雙寫難題 (Dual-Write Trap)*：若先寫入 PostgreSQL 再發送 Kafka，萬一中途斷電就會導致「訂單成立卻沒派單」！
  - *標準解法 —— **Transactional Outbox 模式***：
    1. 在**同一個本地 ACID 交易**中，同時寫入 `orders` 表與 `outbox_events` 表。
    2. 背景 Relay 行程再可靠地將 `outbox_events` 推播至 Kafka，搭配消費端冪等處理達成零掉單！

# 觀念檢核測驗 5 (CCQ 5)

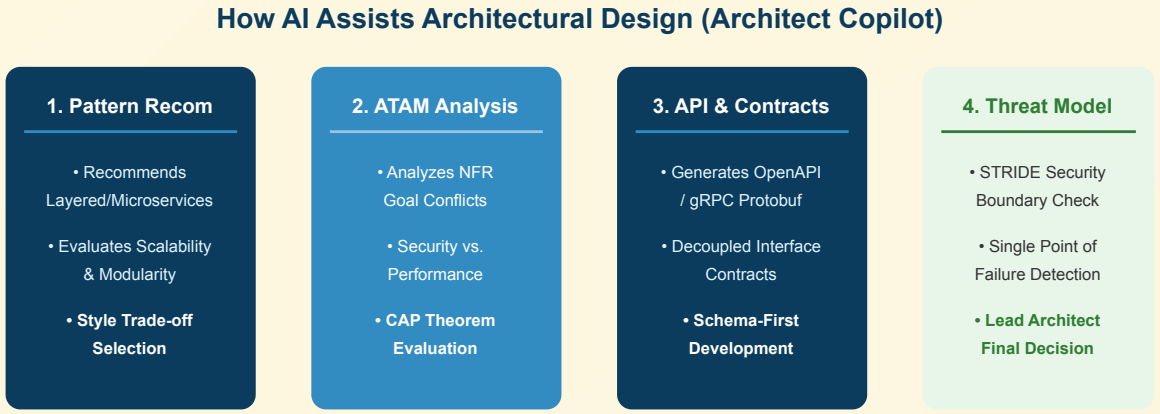
在設計流程的第四階段（**資料與持久化設計**）中，為什麼外送平台將 `orders` 與 `payments` 儲存在 **PostgreSQL (關聯式 OLTP)**，卻將 5 萬名外送員的即時 GPS 座標儲存在 **Redis (`GeoHash` + `TTL = 30s`)**？

- **A.** PostgreSQL 缺乏資料表主鍵支援，而 Redis 則強制執行第三正規化外來鍵約束。
- **B.** Redis 用於永久歸檔歷史財務帳本，而 PostgreSQL 會在 30 秒後自動清除未付款訂單。
- **C.** PostgreSQL 保障帳務 ACID 交易完整性，而記憶體內 Redis 能以 `< 5ms` 處理高頻空間查詢。
- **D.** 將外送員即時 GPS 串流直接寫入 PostgreSQL 可完全消除手機端的無線網路延遲。

# 5.7 AI 賦能軟體設計與盲點防護

> *"AI 不會取代軟體架構師；但紮根嚴謹工程原則並善用 AI 的架構師，將會取代不用 AI 的人。"*

# AI 副駕駛如何賦能「四階段軟體設計流程」？




- **1. 賦能階段一（架構設計 — 5.3）**：模擬 **ATAM 極端故障場景**（*「若晚餐尖峰 Redis 分片當機，派單管線如何降級？」*）、草擬 **ADR 決策記錄**，並自動生成 **UML 套件圖/部屬圖** 之 PlantUML 代碼。
- **2. 賦能階段二（介面與契約 — 5.4）**：自動生成 **OpenAPI 3.1 / Protobuf** 契約，並推導 **DbC 前置條件、後置條件與邊界測試案例**。
- **3. 賦能階段三（組件與 OO 設計 — 5.5）**：在 PR 合併前自動掃描上帝類別與循環相依，將僵化的 `switch/case` 重構為 **SOLID 與 GoF 策略模式**。
- **4. 賦能階段四（資料設計 — 5.6）**：將領域模型轉譯為 **3NF SQL DDL**、建議複合索引 (`«idx»`)，並偵測 ORM `N+1` 查詢效能陷阱。

# 人機協同防護：防範 AI 架構盲點 (Architectural Blind Spots)

- **盲目採納 AI 產生的三大致命風險：**
  - **過度複雜化 (Resume-Driven Architecture)**：AI 經常過度工程化，為 3 個人兩週就能搞定的內部工具推薦 Kubernetes、微服務與 Kafka！
  - **組織脈絡盲目 (Context Blindness)**：AI 完全不知道你們團隊無人會除錯分散式競態條件，也無視你們的雲端預算上限。
  - **膚淺表面重構 (Superficial Refactoring)**：僅替換變數名稱，卻未根本解決資料耦合與架構瓶頸。
- **系統架構的至高金科玉律：**

> ***「AI 提案得失，架構師裁奪決策！」***
> *人類工程師必須依據現實組織環境做裁量，並對系統的長期健全負起終極責任。*

# 5.8 核心概念統整與參考文獻

> *"簡單是可靠性的絕對前提。"*
> *— Edsger W. Dijkstra (圖靈獎得主)*

# 本章核心精華統整矩陣 (Curriculum Synthesis Matrix)

| 章節與設計階段 | 核心設計概念與專屬 UML / 資料模型 | 最關鍵工程啟發 Heuristic |
|---|---|---|
| **5.1 設計流程與規格** | 四階段設計流程、**SDD 與 ADR** | 設計階段修缺陷僅 $1（上線後 $100）；以 SDD 記 How、ADR 記 Why。 |
| **5.2 通用設計原則** | **高凝聚與低耦合** (UML 結構對照) | **極大化凝聚度，極小化耦合度！** 嚴格隱藏模組私有實作細節。 |
| **5.3 階段一：架構設計** | 六大架構模式、ATAM、**套件圖與部屬圖** | 真實系統皆為複合模式；以套件圖控管無環相依、部屬圖規劃雲端容錯。 |
| **5.4 階段二：介面契約** | **契約式設計 (DbC) 與 UML 組件圖** | 明確定義 `Pre`、`Post` 與 `Invariant`；以棒棒糖 (`—o`) 與插座 (`—(`) 熱插拔。 |
| **5.5 階段三：OO 設計** | GoF 23 模式、**SOLID 與 UML 類別圖** | 依賴抽象而非細節 (DIP)；子類別必須嚴守父類別 DbC 契約 (LSP)。 |
| **5.6 階段四：資料設計** | **三層次 ERD**、3NF 與多模態儲存 | 金流訂單用 SQL ACID、高頻座標用 Redis TTL、跨庫同步用 Outbox + Kafka。 |
| **5.7 AI 賦能設計** | 橫跨四階段之 AI 副駕駛典範 | AI 協助發散探索與草擬規格；人類架構師把關組織脈絡與最終責任！ |

# 經典學術與業界參考文獻 (References)


- **軟體架構與設計規格權威文獻：**
  - Len Bass, Paul Clements, Rick Kazman (SEI): *Software Architecture in Practice*, 4th Edition, Addison-Wesley.
  - Bertrand Meyer: *Object-Oriented Software Construction* (契約式設計 Design by Contract 創立著作), Prentice Hall.
  - IEEE Std 1016: *IEEE Standard for Information Technology—Systems Design—Software Design Descriptions (SDD)*.
  - Michael Nygard: *Documenting Architecture Decisions (ADR)*, 2011.
- **經典設計原則、UML、資料設計與設計模式著作：**
  - Robert C. Martin (Uncle Bob): *Clean Architecture: A Craftsman's Guide to Software Structure and Design*, Prentice Hall.
  - Erich Gamma, Richard Helm, Johnson, Vlissides: *Design Patterns: Elements of Reusable Object-Oriented Software (GoF)*, Addison-Wesley.
  - Martin Fowler: *UML Distilled* & *Patterns of Enterprise Application Architecture*, Addison-Wesley.
  - Martin Kleppmann: *Designing Data-Intensive Applications (DDIA)*, O'Reilly Media.

# 感謝聆聽，歡迎提問與深入交流！

## Q & A · 共同討論

**薛念林 教授**
資訊工程學系
逢甲大學